Resource-Constrained Neural Monte Carlo Tree Search for Quantum Boolean Oracle Synthesis

Zixiang Zhou

July 9, 2026

Abstract

Boolean oracle synthesis embeds classical predicates into quantum algorithms, but direct algebraic-normal-form constructions can be costly in non-Clifford gates, CNOTs, logical depth, and temporary workspace. We formulate bit-flip Boolean oracle synthesis as a resource-constrained search problem over ANF/FPRM term sets and introduce Resource-NMCTS, a logical-layer workflow that combines neural action priors, Monte Carlo tree search, Boolean-ring actions, frontier controllers, Pareto archives, and baseline-preserving guards. Unlike SSHR, the method does not search parallelotope candidates; SSHR is used only as a CNOT-oriented small-function baseline. On 177 traditional functions with $n \leq 6$, Pareto-Resource-NMCTS reduces mean T-count by 72.25% and mean weighted score by 67.80% relative to direct ANF synthesis, and its score win/loss/tie counts against ESOP beam, weighted ESOP-MILP, and SSHR-H are 174/0/3, 167/3/7, and 173/4/0. External probes show corresponding score wins against ROS-style LUT (309/0/0), mockturtle (166/11/0 and 64/0/0), CirKit (177/0/0 and 64/0/0), and legacy RevKit CLI exact-oracle synthesis (173/0/4). These comparisons also expose the expected tradeoffs: CirKit is often shallower, and RevKit uses fewer auxiliary lines on most small functions. In the phase branch, a rank-trained shortlist scores 512 of 8192 affine forms per held-out $n = 6$ function and matches the wide-search mean within 0.01% under the $T/R_z = 30$ proxy. For larger term-set instances, a sparse depth-2/4 frontier matches the measured depth-2/3/4 frontier while reducing evaluation time by 24.94–28.88%, and a learned depth-4 gate preserves sparse-frontier score on a 144-pair multi-seed $n = 24, 28, 32, 40$ audit while saving 13.43%. Correctness is supported by 15,294 symbolic, phase-search, and policy-pruning rows, plus 400/400 complete truth table-bridge rows for $n = 21$–25. The contribution is a verified logical-layer search method with strong T-count and weighted-score advantages, not a hardware-mapped, depth-only, or universal-dominance claim.

1 Introduction

Given a Boolean function $f : \{0,1\}^n \to \{0,1\}$, a bit-flip oracle implements

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \tag{1}$$

Such oracles appear in Grover search, constraint solving, and quantum cryptanalysis. Their cost is not determined only by the number of Boolean operations. In a fault-tolerant or resource-limited setting, the chosen Boolean representation, compute/uncompute order, temporary workspace, and phase realization all affect T-count, CNOT count, logical depth, gate count, and peak ancilla.

Existing synthesis routes provide strong local solutions. ESOP, XAG, LUT and BDD representations reduce different Boolean resources; ROS makes LUT-based oracle synthesis resource-aware; RevKit, mockturtle, CirKit, and ABC provide mature logic and reversible-synthesis toolchains; and SSHR uses parallelotope structures in the Boolean hypercube to target small-scale CNOT-oriented synthesis [6, 5, 4, 15, 1, 13, 12, 17, 11, 10, 9]. These methods motivate

the baselines used in this paper, but they also expose a gap: a fixed representation or a single resource objective may miss useful tradeoffs when T-count, CNOTs, depth, gates, and ancilla are optimized together.

We address this gap by treating Boolean oracle synthesis as an explicit search problem. The state is an ANF/FPRM term set; actions are algebraic rewrites that can be expanded back to GF(2) semantics; neural policies and MCTS allocate the search budget; and guard rules ensure that learned candidates do not replace a stronger deterministic baseline. The resulting circuit is checked at the plan level, at the emitted X/CNOT/MCT symbolic level, and, for bridge instances, by full truth table oracle simulation.

The paper makes four contributions. First, it defines Resource-NMCTS as a resource-constrained ANF/FPRM search pipeline rather than as an SSHR extension. Second, it combines Boolean-ring linear-factor screening, neural action priors, MCTS, depth-frontier policies, Pareto archives, and baseline-preserving guards into one logical-layer synthesis workflow. Third, it evaluates the method against ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, CirKit, RevKit CLI, and RevKit phase/Rz baselines on matched functions. Fourth, it separates algorithmic gains from engineering guards through ablations and high-dimensional verification bridges.

Scope and assumptions. The study is intentionally limited to logical-layer Boolean-oracle synthesis. All reported resources are computed before hardware mapping, routing, native-gate scheduling, noise modeling, and magic-state-factory accounting. The weighted score in Eq. (3) is used only to rank verified candidates; T-count, CNOT count, depth, gate count, and peak ancilla are also reported separately, and cross-toolchain comparisons are interpreted through the baseline comparability audit rather than as a universal leaderboard.

Table 1: Contribution-to-evidence map. Each contribution is tied to concrete implementation mechanisms, paper evidence, and the boundary that prevents the claim from being overstated.

Contribution	Implementation	Paper evidence	Boundary
Resource-constrained Boolean-oracle search formulation	ANF/FPRM term-set state, Boolean-ring and affine actions, guarded candidate selection, symbolic circuit emission.	Fig. 1; Table 2	Logical-layer oracle synthesis only; no routing, native-gate mapping, or noise model.
Neural/MCTS resource-search workflow	Neural action priors, MCTS/beam/portfolio search, Pareto archive, frontier policies, staged and sparse learned gates.	Tables 23, 24, 33; Fig. 5	The largest gains come from searchable algebraic actions plus guarded selection; the paper does not claim deep RL alone explains all gains.
Broad baseline and toolchain comparison	Direct ANF, ESOP beam/MILP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, CirKit, RevKit CLI, and phase/Rz probes.	Tables 6, 7, 19; Fig. 3	Strong T-count and weighted-score evidence, not universal CNOT/depth/ancilla dominance or full ROS reproduction.
High-dimensional and phase-search verification envelope	$n = 20$–40 symbolic term-set checks, $n = 21$–25 complete truth-table bridges, Affine-FPRM phase search, learned phase shortlist.	Tables 39, 40, 38; Fig. 7	Large rows are logically and symbolically verified; complete truth-table enumeration remains limited to bridge slices.

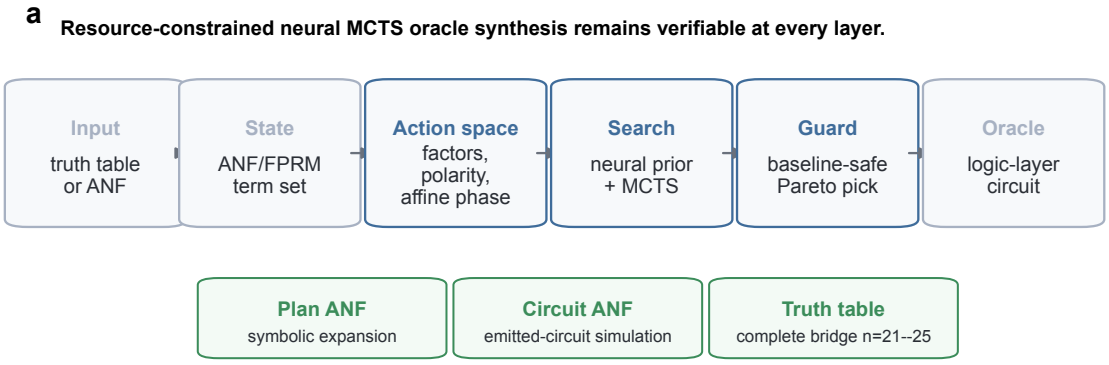

Figure 1: **Verified search pipeline.** Resource-NMCTS operates on complete truth table inputs or ANF term sets, searches algebraic actions under neural and tree-search control, and verifies candidate circuits at plan, emitted symbolic-circuit, and complete-truth table levels.

2 Related Work

2.1 Boolean representations for oracle synthesis

Low-T oracle synthesis is closely tied to the Boolean intermediate representation. BDD-based reversible synthesis uses symbolic decision diagrams to handle large functions [15]. ROS frames oracle synthesis as resource-constrained LUT mapping with downstream reversible implementation costs [6]. XAG-based compilation separates XOR structure from AND nodes, making multiplicative complexity a proxy for non-Clifford cost [5, 4]. Back-end-aware oracle synthesis further connects logic synthesis to downstream resource metrics, but that line includes mapping concerns that are outside the scope of this logical-layer study [16].

2.2 Reversible and logic-synthesis toolchains

ABC supplies mature AIG, XAG/GIA, LUT, and ESOP optimization flows for classical logic [1]. mockturtle and CirKit provide reusable logic-network optimization libraries and shells in the EPFL logic-synthesis ecosystem [13, 10, 9]. RevKit provides reversible-logic and quantum-compilation functionality, including the Python `oracle_synth` API and legacy command-line reversible synthesis flows [12, 11]. In this work these toolchains are used as external baselines or probes. They are not called inside Resource-NMCTS, which keeps the proposed search policy separated from the comparison backends.

2.3 Learning-guided synthesis

Learning-guided synthesis has been explored for both classical and quantum circuits. ShortCircuit uses AlphaZero-style search for classical Boolean circuit design [14]. Gumbel AlphaZero has been used for Clifford+T unitary synthesis [7]; reinforcement learning has been used for non-Clifford-count optimization and ZX-calculus rewriting [2, 8]; MCTS has been used for quantum circuit transformation under connectivity constraints [18]; and MonteQ uses MCTS for Hamiltonian-simulation circuit synthesis [3]. The present work is different in scope: it is a logical-layer Boolean-oracle synthesizer, not a hardware-routing method, not a Hamiltonian-simulation scheduler, not a gate-by-gate unitary generator, and not an SSHR parallelotope search. Its learned component ranks verifiable Boolean-algebraic actions, while ESOP, BDD, XAG/LUT, ROS-style, SSHR, RevKit, mockturtle, CirKit, and ABC flows define the comparison set.

Table 2 summarizes this positioning. It is used to keep the manuscript's contribution narrow and testable: existing tools and representations define strong baselines or motivation, while Resource-NMCTS is claimed only as a logical-layer resource-aware search method over verifiable Boolean-algebraic actions.

Table 2: Related-work positioning matrix. The table states what each literature family optimizes, which gap motivates the proposed search, and how the manuscript uses that family in experiments or claim boundaries.

Line of work	Representative work	Main lever	Gap for this paper	Manuscript use
BDD-based reversible synthesis	BDD reversible synthesis [15]	Symbolic decision-diagram representation for large Boolean functions.	Representation scalability does not by itself optimize T, CNOT, depth, gates, and ancilla jointly.	Used as a representation-level baseline family, not as the proposed search space.
LUT/ROS-style oracle synthesis	ROS and back-end-aware oracle synthesis [6, 16]	Resource-aware LUT mapping and downstream implementation-cost estimates.	Full ROS garbage-management and hardware/back-end mapping are outside the logical-layer claim.	Reproduced through a verified ROS-style LUT proxy and line-aware reselection stress tests.
XAG and multiplicative complexity	XAG compilation and multiplicative-complexity oracle cost [4, 5]	Separates XOR-linear structure from AND nodes as a non-Clifford proxy.	A fixed network optimization flow may miss algebraic FPRM/ANF search actions and Pareto tradeoffs.	Used through ABC, mockturtle, and CirKit probes plus exact small XAG lower-bound checks.
Logic and reversible toolchains	ABC, EPFL libraries, RevKit, mockturtle, and CirKit [1, 13, 12, 11, 10, 9]	Mature optimization passes and reversible-synthesis flows.	Tool output defines strong baselines but does not expose the neural/MCTS algebraic action policy.	Used as external probes and exact-oracle reversible-synthesis comparisons.
SSHR geometric synthesis	Parallelotope-based CNOT-oriented synthesis [17]	Small-scale Boolean hypercube geometry targeting CNOT-oriented covers.	The objective is CNOT-oriented and small-function focused, not a general resource-weighted AI search.	Used as a CNOT-oriented small-function baseline; not used by the proposed method.
Learning-guided circuit synthesis	AlphaZero, reinforcement learning, MCTS, and ZX-guided synthesis [14, 7, 2, 8, 18, 3]	Learns or searches over circuit-design and circuit-transformation actions.	Most targets are classical circuits, unitary synthesis, routing/transformation, ZX rewriting, or Hamiltonian simulation rather than Boolean-oracle ANF/FPRM term search.	Motivates neural/MCTS control while keeping actions verifiable at the Boolean-algebraic oracle layer.

3 Problem and Resource Model

We represent a Boolean function by its square-free ANF

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \tag{2}$$

where each monomial m is stored as a bit mask. A direct construction applies one controlled action per monomial. The synthesis problem is to replace this direct construction with reusable compute/action/uncompute subplans that implement Eq. (1) while lowering a multi-resource objective.

All resource numbers in this paper are logical-layer estimates over X, CNOT and multi-controlled Toffoli gates. Candidate selection uses

$$\text{score} = T + 0.04\,\text{CNOT} + 0.015\,\text{depth} + 0.01\,\text{gates} + 2\,\text{ancilla}. \tag{3}$$

The score is a ranking objective, not a physical runtime or error model. We therefore report T-count, CNOT count, depth, gate count, and peak ancilla separately whenever a tradeoff is relevant.

4 Method

4.1 Search state and actions

The Resource-NMCTS state is the current set of unsynthesized ANF terms. An action chooses a reusable algebraic structure, synthesizes that structure into a temporary line, uses it to control a quotient subproblem, and then uncomputes it. Candidate actions include common monomial factors, FPRM polarity rewrites, CNOT-computed linear parity factors, Boolean-ring linear factors, and bounded affine changes of variables. Each action is scored by its immediate resource estimate and by the size and structure of the induced subproblems.

Table 3 summarizes the complete execution path. The central invariant is that learning and tree search only control which verifiable algebraic candidates are evaluated or selected; semantic correctness is supplied by the ANF, emitted-circuit, truth-table, or phase verifier attached to the corresponding stage.

Table 3: End-to-end synthesis and verification workflow. Each stage records the mechanism used by Resource-NMCTS, the invariant that prevents the learned search from becoming a semantic oracle, and the artifact produced for downstream analysis.

Stage	Mechanism	Invariant	Artifact
Input normalization	Convert a truth table, exported benchmark, or supplied term set into a square-free ANF monomial mask set $\mathcal{M}$.	The term set must evaluate to the target Boolean function before any search step is trusted.	ANF terms, preset name, and source manifest.
Candidate generation	Generate direct ANF, FPRM polarity, common-factor, Boolean-ring linear-factor, affine, and depth-frontier candidates.	Every action must have an explicit GF(2) expansion back to the original variables.	Candidate logical plans with resource estimates.
Search control	Use neural action priors, MCTS/beam expansion, staged frontiers, and sparse learned gates to allocate the search budget.	Learned decisions rank or skip evaluations; they do not define semantic correctness.	Scored candidate pool and search-time diagnostics.
Guarded selection	Compare learned and expensive candidates with deterministic baselines, then apply Pareto filtering before active-score selection.	A selected plan must be a valid generated candidate and remains bounded by the stated guard and score model.	Selected logical plan and non-dominated archive entry.
Circuit emission	Expand the selected plan into X/CNOT/MCT compute-action-uncompute operations and compute logical resource counts.	The reported score is a ranking objective; T, CNOT, depth, gates, and ancilla are retained separately.	Logical circuit proxy and resource row.
Verification and reporting	Check the plan algebraically, check emitted-circuit ANF semantics, add full truth-table or phase checks where feasible, and write raw CSV plus manifest rows.	Paper comparisons use matched rows that pass the relevant semantic checks and exclude errors, skips, and timeouts.	`raw_*.csv`, `summary_*.csv`, `manifest_*.json`, and paper tables.

Table 4 gives the algorithmic contract used by the implementation. It is written as a source-anchored contract rather than as language-only pseudocode: each row identifies the implementation anchors that make the corresponding method statement executable and auditable. This is important for the learning components, because the neural scorer and learned frontier gates are only allowed to change ranking, budget allocation, or skipping decisions; correctness and final resource reporting remain attached to explicit algebraic expansions, guarded candidate selection, and semantic verification.

Table 4: Algorithm contract for Resource-NMCTS. The table converts the method description into implementation-anchored operational rules and reviewer-facing guarantees.

Stage	Operational rule	Source anchors	Reviewer-facing guarantee
Canonical state	Normalize a truth table, exported benchmark, or supplied term list into a square-free ANF monomial set; direct ANF remains the fallback plan at every state.	synthesizers.py: anf_monomials; factor_plan.py: frozenset, direct_plan	The search starts from the same Boolean function as the baseline and always has a syntactically valid logical oracle construction.
Action generation	Generate monomial factors, linear/Boolean-ring factors, FPRM polarity actions, and affine/linear-pair candidates only when they have an explicit GF(2) expansion.	factor_plan.py: candidate_actions, linear_factor_actions, _linear_boolean_expansion; synthesizers.py: affine_linear_pair_beam_plan	Learning never invents a semantic rule; it can only rank or select among algebraic actions that the emitter and verifier can expand.
Prior-guided tree search	Use the neural scorer to bias action priors, then run PUCT/MCTS recursion with greedy rollouts and bounded candidate_top_k expansion.	neural_policy.py: NeuralScorer, score_many; nmcts_solver.py: NeuralMCTSSolver, c_puct; factor_plan.py: candidate_top_k	The neural component is a budget-allocation prior, not a correctness oracle or an unrestricted circuit generator.
Baseline guard	Evaluate deterministic, beam, MCTS, neural, and screen candidates as a portfolio; guarded neural variants return the better of the baseline and learned candidate.	synthesizers.py: _run_child_portfolio, min([baseline, neural_plan], _portfolio_result, direct_anf	A learned or expensive branch is allowed to help but is not allowed to replace a stronger verified deterministic candidate in guarded rows.
Pareto archive	Collect candidates from multiple resource profiles, remove candidates dominated in T, CNOT, depth, gates, and peak ancilla, then select by the active score.	synthesizers.py: _pareto_front, _dominates, _resource_selection_key, pareto_resource_nmcts	Weighted-score wins are kept separate from multi-resource dominance, which is why tradeoff tables remain part of the Results section.
Large-scale controllers	Use depth-frontier policies and sparse gates to decide which high-dimensional Boolean-ring screens to evaluate, with deterministic sparse/full frontiers kept as references.	run_truth_bridge_terms.py: load_depth2_guard, load_frontier_policy, depth_frontier_policy, screen_depth4	High-dimensional controllers are planning-time controls over symbolic term-set verification, not hardware schedulers or exhaustive truth-table solvers.
Emission and verification	Emit X/CNOT/MCT compute-action-uncompute circuits, verify ANF semantics symbolically, and use complete truth-table or phase checks where feasible before writing raw result rows.	factor_plan.py: emit_plan_to_circuit, verify_plan_anf, verify_circuit_anf, verify_oracle; run_experiments.py: correct	Paper comparisons consume matched rows that passed the relevant semantic check; skipped, errored, or timed-out rows are not silently counted.

Table 5 records the corresponding search-budget contract. The table is intentionally operational: it states the configured candidate fan-out, auxiliary-line, MCTS, polarity-search, portfolio, Pareto, frontier-controller, and verification caps that make the method rerunnable. These caps are part of the claim boundary. They explain why the method scales as a bounded logical-layer search workflow while also making clear that the reported candidates are not certificates of global optimality.

Table 5: Search-budget contract for Resource-NMCTS. Each row states the explicit cap used to make the search rerunnable, the scalability role of that cap, and the limitation it leaves in place.

Budget family	Explicit cap	Scalability role	Boundary
Action fan-out	candidate_top_k=24; max_factor_size=5; min_factor_count=2	Bounds the number of algebraic actions expanded at each ANF state.	The cap controls search cost; it is not a proof that the globally best factor action is always included.
Ancilla and recursion	max_factor_ancilla=4; depth guard=64 in MCTS recursion	Prevents recursive factoring from allocating unbounded temporary factor lines.	This is a resource guard, not an ancilla-optimality theorem.
PUCT/MCTS simulations	mcts_simulations=96; neural_mcts_simulations=128; c_puct=1.35	Makes tree-search compute an explicit, rerunnable budget rather than an open-ended optimizer.	The MCTS budget supports a bounded search-control claim, not exact optimal synthesis.
Polarity and affine search	max_polarities=384; high-dimensional direct screening narrows expensive planning trials	Keeps fixed-polarity and affine preconditioning searches tractable on larger sparse term sets.	Screening is a controlled heuristic; missed polarities remain part of the search-boundary limitation.
Resource portfolio	fast_config caps candidate_top_k$\leq$18, MCTS$\leq$32/40, max_polarities$\leq$16/24 before portfolio selection	Separates cheap deterministic candidates from more expensive MCTS or affine branches.	Portfolio selection gives a best verified candidate among attempted children, not a certificate over all possible children.
Pareto archive	Deduplicate by (T,CNOT,depth,gates,peak ancilla), remove dominated candidates, then rank the non-dominated front by active score	Keeps multi-objective diversity without carrying every generated candidate into the final comparison.	The archive is Pareto over generated candidates only; it is not a global Pareto frontier.
High-dimensional frontier controllers	Staged, sparse, and learned gates decide whether depth-3/depth-4 Boolean-ring screens are evaluated	Moves large-term-set runs from all-frontier evaluation toward validated budget control.	The controller is empirical on the audited slices and is not a proof that depth-4 can always be skipped.
Verification route	Symbolic ANF/circuit checks for large rows; complete truth-table bridge slices where enumeration is feasible	Allows high-dimensional evidence without pretending that every large instance was exhaustively enumerated.	Symbolic verification is exact for emitted GF(2) semantics but does not replace hardware mapping or exhaustive enumeration for all n.

4.2 Boolean-ring linear factors

A key extension is allowing the linear factor and the quotient to share variables under Boolean-ring semantics. For example,

$$(x_i \oplus x_j)x_i m = x_i m \oplus x_i x_j m, \tag{4}$$

because $x_i^2 = x_i$ over the Boolean ring. Circuit-wise, the parity $x_i \oplus x_j$ is computed by CNOTs into a temporary line, used as a control for the quotient plan, and uncomputed. This expands the algebraic action space without changing the GF(2) verifier.

4.3 Neural prior, MCTS, frontier policy, and guard

The neural action prior ranks candidate actions using term-count, degree, coverage, variable-support, and immediate-resource features. MCTS then explores combinations of actions with deterministic rollout estimates. A separate frontier policy selects among depth-2, depth-3, and depth-4 Boolean screening frontiers; a cost-aware version trades a small score increase for lower planning time and reduced ancillary lifetime area. We also use two conditional frontier controllers. The staged controller evaluates depth-2 and depth-3 screens first and only runs depth-4 when the measured depth-3 improvement is large enough. The sparse depth-4 gate instead uses the depth-2 state to predict whether the deterministic sparse depth-2/4 frontier needs its depth-4 evaluation; its threshold is selected on validation data to avoid false skips of improving depth-4 cases. Finally, a guard compares learned or expensive candidates to deterministic baselines and

returns the best valid candidate under Eq. (3). This makes learning a search-control component, not the source of semantic correctness.

4.4 Pareto archive

The Pareto variant evaluates candidates under several internal objectives, including active-score, T-sparse, CNOT/depth-heavy, gate-sensitive, and ancilla-tight rankings. It removes candidates dominated simultaneously in T-count, CNOT count, depth, gate count, and peak ancilla, then selects from the remaining archive using the active score. The archive is used only after each candidate has passed the same plan and emitted-circuit checks as the base method.

4.5 Phase/Rz branch

RevKit `oracle_synth` returns phase/Rz netlists rather than direct bit-flip Clifford+T circuits. To avoid treating that boundary only as a cost sensitivity, we also implement a phase-parity emitter: each ANF monomial is expanded into merged parity-phase gadgets and verified up to global phase. Fixed-polarity FPRM search and bounded affine preconditioning are then added as phase-search actions. This branch is reported as a logical phase/Rz proxy; it does not output approximate rotation sequences or mapped hardware circuits.

5 Experimental Design

Experiments answer four questions. First, does Resource-NMCTS reduce T-count and weighted score on small functions where full truth table checking is practical? Second, do the conclusions survive comparisons with ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, CirKit, RevKit CLI, and RevKit phase/Rz baselines? Third, which components–affine search, MCTS, neural priors, guards, frontier policies, and Pareto archives–account for the gains? Fourth, how far can correctness evidence be pushed beyond small truth tables?

The traditional slice contains 177 functions with $n \leq 6$. High-dimensional logic-network probes include exported $n = 14$, $n = 15$, $n = 16$, and $n = 18$ random-ANF sets for ABC/BDD, and $n = 14$ sets for mockturtle and CirKit. Scale and bridge experiments cover item-level $n = 20$–40 term sets and complete truth table bridge functions at $n = 21$–25. Paired comparisons include only matching functions or items with successful semantic checks and no timeout. Because these baselines operate at different abstraction levels, Table 6 first separates their role in the argument from the claims they cannot support. Table 7 then summarizes the quantitative comparison evidence for each baseline family.

Table 6: Baseline claim matrix. The table explains why each comparison family is included, what conclusion it supports, and which stronger conclusion is outside the scope of the logical-layer experiments.

Role	Compared methods	Why it matters	Supported claim	Excluded claim
Primary resource-efficiency baselines	Direct ANF, AND-direct ANF, ESOP beam/MILP, BDD/ABC-ESOP, SSHR-H/SSHR-I	These methods solve the same bit-flip Boolean-oracle task under the paper's logical resource model.	Resource-NMCTS improves T-count and weighted score on matched small functions while SSHR remains a strong CNOT counterpoint.	Does not prove universal CNOT, depth, ancilla, or hardware-level optimality.
External logic-network probes	ABC AIG/XAG/LUT, ROS-style LUT proxy, mockturtle KLUT-to-XAG, CirKit AIG/MC	They test whether mature Boolean-network optimization already removes the apparent advantage over direct algebraic baselines.	The score advantage persists against stronger exported-network and official-header tool probes.	Does not reproduce full ROS SAT garbage management or reversible/hardware mapping.
Exact reversible-synthesis probe	Legacy RevKit CLI TBS/DBS/RMS exact-oracle portfolio	It embeds each function as $(x, y) \mapsto (x, y \oplus f(x))$ and checks a genuine reversible-synthesis toolchain.	Resource-NMCTS has lower T-count and weighted score against this portfolio but pays more auxiliary lines.	Does not imply lower line count, routed depth, or mapped Clifford+T cost.
Phase/R_z branch	RevKit oracle_synth, phase-parity ANF, fixed-polarity FPRM, affine FPRM, learned phase shortlist	It tests whether the same search framing extends to phase-oracle and affine-preconditioned R_z cost proxies.	Affine and learned candidate pruning improve verified logical phase/R_z proxy objectives.	Does not output approximate rotation sequences or a final Clifford+T decomposition.
Internal search ablations	No-MCTS portfolio, learned prior off, guard off, Pareto archive off, depth-frontier variants, sparse gate	They separate representation effects from the contribution of search, neural ranking, guards, and budget control.	AI-guided search and guards add measurable gains or time savings beyond deterministic construction.	Does not support a claim that deep reinforcement learning alone causes the whole improvement.
Scaling and correctness bridges	$n = 20$–40 term-set symbolic runs; $n = 21$–25 complete truth-table bridge rows	They push semantic checks beyond the small truth-table slice while keeping full verification where still feasible.	The emitted logical-layer circuits remain symbolically and bridge-truth-table verified at larger dimensions.	Does not make exhaustive high-dimensional truth-table benchmarking feasible or complete.
Trade-off audits	Raw multi-resource dominance, line-aware LUT reselection, schedule proxy, sparse-threshold sweep	They prevent a weighted-score result from hiding CNOT, depth, line-count, or planning-time tradeoffs.	The method occupies a strong T/score point with explicit depth, CNOT, ancilla, and runtime boundaries.	Does not justify reporting a single-metric victory as complete dominance.

Table 7: Reviewer-facing evidence matrix. The table consolidates the baseline family, coverage, verification count, headline paired result, and scope boundary used by the manuscript claims.

Evidence block	Scope	Verified rows	Main score result	Boundary
Internal logical baselines	$n = 3$–6	1770/1770	Pareto vs direct ANF 172/1/4, -67.80%; vs ESOP-MILP 167/3/7, -29.84%	Same verifier/cost model; includes direct, AND-direct, ESOP beam/MILP, SSHR-H, MCTS, affine, and Pareto variants.
Exported SSHR/ABC/BDD extension	$n = 3$–6	1416/1416	Resource-NMCTS vs SSHR-I CNOT 172/5/0, -29.48%; vs ABC-XAG 177/0/0, -64.15%	SSHR-I rows use an 8 s Gurobi budget; ABC/BDD rows are logical estimates over exported truth tables.
ROS-style LUT proxy	$n = 3$–6, 14, 15, 16, 18	309/309 best rows; 927/927 K-sweep rows	Pareto vs best-K proxy 309/0/0, -84.27%	Verified LUT proxy only; no official ROS SAT garbage management, reversible emission, or hardware mapping.
mockturtle KLUT-to-XAG probe	$n = 3$–6, 14	241/241	$n \leq 6$ 166/11/0, -31.50%; $n = 14$ 64/0/0, -91.49%	Official-header XAG resynthesis probe; still a logical proxy, not full ROS or reversible garbage management.
CirKit AIG/MC probe	$n = 3$–6, 14	241/241	$n \leq 6$ 177/0/0, -62.34%; $n = 14$ 64/0/0, -94.46%	AIG multiplicative-complexity probe; strongest current depth-oriented external counterpoint.
Legacy RevKit CLI exact oracle	$n = 3$–6	708/708	Pareto vs best-score portfolio 173/0/4, -67.28%	Exact reversible oracle permutation; CNOT/depth are derived from Toffoli-control distributions and ancilla is a visible tradeoff.
RevKit phase/Rz branch	$n = 3$–6	531/531	Affine phase vs RevKit oracle_synth 177/0/0, -65.50%	Logical phase/Rz proxy verified up to global phase; not an approximate-rotation or final Clifford+T sequence.
Learned phase pruning	$n = 3$–6	7611/7611	Held-out n=6 rank-diverse top512 vs wide128 0/7/31, +0.00%	Policy-pruned affine candidate ranking; evidence is held-out exact scoring, not a standalone compiler backend.
High-dimensional frontier search	$n = 20$, 24, 28, 32, 40	1608/1608	Stage-gated vs all-depth scale 0/4/92, +0.04% score; -25.43% time	Term-set symbolic verification with emitted-circuit ANF checks; depth frontier is a planning guard, not a hardware scheduler.
Complete truth-table bridges	$n = 21$–25	400/400	Bridge rows verify plan, emitted symbolic circuit, and complete truth table for $n = 21$–25.	Bridge set is intentionally narrow because complete truth tables grow exponentially.

Table 8 makes the comparison protocol explicit as reviewer-facing questions. It is included to prevent the baseline set from being read as a single leaderboard: each layer has a usable conclusion and a stronger conclusion that the evidence does not support.

Table 8: Comparison protocol audit. Each comparison layer is tied to the reviewer question it answers, the conclusion supported by the current evidence, and the stronger claim excluded by the logical-layer experiment design.

Comparison layer	Reviewer question	Usable conclusion	Excluded conclusion
Primary same-task Boolean-oracle baselines	Is the method only compared with weak or self-written constructions?	Matched bit-flip oracle baselines support lower T-count and weighted score under the paper's logical model.	They do not prove universal CNOT, depth, ancilla, line-count, or hardware-mapped optimality.
External logic-network and LUT/XAG/AIG probes	Does the advantage survive mature external logic-synthesis toolchains?	External probes test whether the score advantage persists beyond the project's internal baselines.	They are not a full ROS SAT garbage-management flow, reversible emission, or hardware-mapped comparison.
Exact reversible-synthesis counterpoint	Is there a reversible-synthesis comparison, not only logic-network proxies?	RevKit CLI supports a genuine exact-oracle reversible-synthesis probe with lower T/score for Resource-NMCTS.	It does not imply lower auxiliary-line count, routed depth, or mapped Clifford+T cost.
Phase/R_z proxy branch	How is the RevKit phase/R_z gate-set mismatch handled?	The phase branch supports a logical R_z/phase proxy and learned affine shortlist result.	It is not a final approximate-rotation sequence or Clifford+T decomposition.
Internal search-control ablations	Is the AI/MCTS contribution separated from the algebraic search space?	Ablations support bounded search-control, pruning, gating, and planning-time gains.	They do not show that deep reinforcement learning alone causes the main resource drop.
Scaling and correctness bridges	Does the work go beyond small truth-table functions while preserving correctness evidence?	Large rows support logical symbolic correctness and bridge-truth-table verification within the stated envelope.	They do not prove exhaustive high-dimensional truth-table optimality or benchmarking completeness.
Trade-off and non-dominance audits	Does the weighted score hide unfavorable CNOT, depth, ancilla, or runtime behavior?	The paper can claim a strong T/weighted-score point while explicitly reporting unfavorable resource dimensions.	It cannot report a single-metric score win as complete Pareto or hardware dominance.

Table 9 makes the fairness boundary explicit. It separates comparisons that share the same bit-flip oracle target from logic-network proxies, exact reversible-oracle probes, phase/R_z proxies, and large symbolic bridge checks. The table is intentionally conservative: a comparison is used only for the conclusion supported by its task alignment, verification route, and resource abstraction.

Table 9: Baseline comparability audit. The table records why each comparison family is fair enough for the paper's bounded claim, which controls are used, and what residual abstraction mismatch remains.

Baseline family	Task alignment	Fairness control	Residual risk	Usable claim
Direct algebraic and ESOP baselines	Same bit-flip Boolean oracle target and the same logical resource model.	Matched $n \leq 6$ functions, complete truth-table checks, identical score coefficients, and paired per-function comparisons.	These baselines are representation-level constructions, not mature external compilers.	Primary evidence for lower T-count and weighted score under the paper's own oracle model.
SSHR-H and SSHR-I	Same small Boolean-function oracle setting, but with a CNOT-oriented parallelotope search objective.	Matched functions and resource extraction; SSHR-I timeout rows are separated from completed rows.	SSHR is optimized for CNOT structure and small n, so it is a counterpoint rather than the method's design parent.	Resource-NMCTS improves score and T-count while SSHR remains the strongest CNOT reference.
ABC/BDD and exported logic networks	Same exported Boolean functions, evaluated through logical AIG/XAG/LUT/BDD resource proxies.	Rows with errors, skips, or failed truth-table checks are excluded; paired comparisons use matched names.	Network-level counts are proxies and do not include reversible garbage management or quantum routing.	Mature classical logic optimization does not by itself remove the T/score advantage.
ROS-style LUT, mockturtle, and CirKit probes	Same benchmark functions are passed through LUT/XAG/AIG toolchain probes and verified after export/readback when available.	The LUT proxy includes best-K and line-aware reselection; mockturtle and CirKit use official toolchain paths and row-level correctness checks.	These are not full ROS SAT garbage-management or hardware-mapped reversible flows.	The advantage persists beyond self-written baselines, with CirKit exposing a real depth tradeoff.
Legacy RevKit CLI exact-oracle portfolio	Each function is embedded as the exact reversible permutation $(x, y) \mapsto (x, y \oplus f(x))$.	TBS, DBS, and RMS are all run and reduced to a per-function best-score portfolio.	CNOT and depth are derived from Toffoli-control distributions; RevKit uses fewer auxiliary lines on most rows.	A genuine reversible-synthesis probe still favors Resource-NMCTS on T-count and score, but not on line count.
Phase/R_z baselines and learned shortlist	Same Boolean functions are evaluated as phase-oracle or affine-FPRM phase-search instances.	Rows are verified up to global phase; learned shortlists are checked against same-budget random controls and wide exact search.	The branch is a logical R_z/phase proxy and does not output rotation-synthesis sequences.	The search framing extends to verified phase/R_z proxies, but it is not a finished Clifford+T compiler.
High-dimensional symbolic and bridge checks	Large term-set instances and $n = 21$–25 bridge functions test the same emitted logical oracle semantics.	Symbolic ANF/circuit checks cover all reported high-dimensional rows; bridge slices add complete truth-table checks.	Complete truth-table enumeration is limited to bridge slices and generated functions.	The large-scale evidence supports semantic correctness and scaling of the logical search, not exhaustive high-dimensional optimality.

Table 10 records the strongest opposing evidence that bounds the comparison claims. This audit is included because a meaningful comparison should not hide metrics on which other methods remain strong. SSHR is therefore treated as a CNOT counterpoint, CirKit as a depth counterpoint, RevKit CLI as an auxiliary-line counterpoint, and the learned-prior rows as evidence for incremental search control rather than for a deep-RL-only explanation.

Table 10: Counterpoint and claim-boundary audit. The table reports the strongest unfavorable metric evidence that constrains the manuscript's claims, the favorable evidence that keeps each comparison meaningful, and the resulting claim boundary.

Counterpoint	Opposing evidence	Favorable evidence	Claim boundary
SSHR family CNOT optimum	vs SSHR-I CNOT: CNOT 0/168/9, +62.06%; vs SSHR-H: CNOT 43/128/6, +18.99%	score vs SSHR-I CNOT 173/4/0, -31.89%; score vs SSHR-H 173/4/0, -41.06%	Use SSHR as a small-function CNOT counterpoint; claim T-count and weighted-score advantage, not CNOT dominance.
CirKit AIG/MC depth	depth 16/156/5, +93.16%	score 177/0/0, -62.34%	Use CirKit as a depth-oriented external probe; do not claim depth dominance over logic-network synthesis.
RevKit exact-oracle auxiliary lines	peak ancilla 0/169/8, +153.11%	score 173/0/4, -67.28%	Use RevKit CLI as an exact reversible-oracle probe; do not claim line-count or mapped Clifford+T dominance.
Learned prior is incremental	time 37/140/0, +18.77% versus no-prior rerun	score 29/0/148, -0.78%	Frame AI as search control, ranking, pruning, and gating; do not claim deep RL alone explains the main resource drop.
Large-n verification is bounded	complete truth-table enumeration is limited to bridge slices	symbolic scale rows 1608/1608; complete truth-table bridge rows 400/400	Claim logical correctness inside symbolic and bridge-verification envelopes, not exhaustive large-dimensional benchmarking.

The comparison set should therefore be read as layered evidence rather than as a single universal leaderboard. Direct ANF, ESOP, ABC/BDD, and related representation baselines test matched bit-flip oracle resource efficiency; SSHR tests whether the proposed search remains competitive against a CNOT-oriented geometric method on small functions; mockturtle, CirKit, ROS-style LUT, and RevKit probes test whether the advantage persists outside the project's own implementations; and internal ablations test whether the search controls contribute beyond deterministic construction. The results below use each family only for the claim it can support.

The experiments were executed as a reproducible artifact package rather than as isolated hand runs. Table 11 records the local compute envelope, manifest-level parallelism, artifact counts, and external toolchain provenance. Runtime numbers should be read within this workstation context; the logical resource counts are the portable experimental outcome.

Table 11: Compute and reproducibility audit. The table documents the workstation, parallel-run manifests, artifact coverage, and external-tool commits used by the current submission package.

Aspect	Evidence	Boundary
Host compute envelope	Apple M4 Pro; 14 physical/14 logical CPU cores; 24.0 GiB RAM; Apple M4 Pro, 20 GPU cores, Metal 4; Python 3.11.15	Workstation context for reproducibility; not a hardware-mapping result.
Learning accelerator	PyTorch 2.12.0; MPS=True; CUDA=False	GPU/MPS is available for neural training; synthesis resources remain logical-layer estimates.
Parallel-run provenance	55 manifests record worker counts; max workers=10; traditional resource 10 workers/1770 rows; RevKit CLI 8 workers/708 rows; ROS-style LUT 8 workers/927 rows; CirKit 8 workers/177 rows; mockturtle 4 workers/177 rows	Wall times are machine-dependent and are not claimed as portable speedups.
Artifact coverage	94 top-level run/train/analyze scripts; 145 raw CSVs; 206 summaries; 105 manifests; 165 paper tables; 7 figure panels (21 PDF/PNG/SVG assets)	Counts describe the current artifact package, not independent datasets.
External-tool provenance	mockturtle 25beb0e; CirKit 4531533; RevKit CLI 104eb35	Toolchain rows are logic-level probes or exact-oracle reversible probes, not full hardware flows.

Table 12 records the claim-to-artifact chain for the submission package. It is not a new result table; its role is to make each headline claim auditable by pointing to the scripts, CSVs, tables, figures, and manuscript anchors that carry the evidence.

Table 12: Submission traceability audit. Each row links a paper-facing claim family to the manuscript location and artifact chain used to support it.

Claim family	Paper-facing use	Manuscript anchor	Boundary
Method formulation	Defines Resource-NMCTS as a source-anchored logical ANF/FPRM search workflow.	Method, Tables method-workflow, algorithm-contract, and search-budget-contract	Establishes executable stages, semantic/resource guarantees, and bounded search budgets, not a hardware compiler or global optimality proof.
Contribution mapping	Links each stated contribution to implementation, evidence, and claim boundary.	Introduction, Table contribution-map	Prevents the introduction from making unsupported novelty claims.
Primary small-function resources	Supports T-count and weighted-score claims on matched $n \leq 6$ functions.	Results, traditional functions	Same logical cost model; not a CNOT-only or hardware-level claim.
Baseline scope and fairness	Explains why each baseline family is comparable and which claim it can support.	Experimental Design, baseline matrices and counterpoint audit	Treats comparisons as layered evidence and reports unfavorable metric counterpoints rather than a universal leaderboard.
External toolchain probes	Traces ROS-style LUT, mockturtle, CirKit, and RevKit CLI comparison evidence.	Results, external probes	Logic-level probes or exact-oracle reversible probe; not full hardware mapping.
Multi-resource tradeoff	Audits whether weighted-score wins imply raw-resource, schedule-proxy, or auxiliary-lifetime dominance.	Results, raw multi-resource dominance and schedule-proxy audit	Dominance uses logical T-count, CNOT, depth, peak ancilla, T-depth proxy, and auxiliary lifetime; no routing or native-gate scheduling is claimed.
Learned-control contribution	Separates search-control baselines and promoted learned controllers from limited diagnostics.	Results, search contribution and ablation	Supports guarded learned-control evidence, not a claim that RL alone explains all gains.
High-dimensional verification	Traces large symbolic checks, phase checks, and complete truth-table bridge slices.	Results, high-dimensional verification	Large rows are symbolic or bridge-slice verified, not exhaustive high-dimensional truth tables.
Archive package integrity	Hashes stable submission payload groups and records archive boundaries.	Experimental Design, archive manifest	Excludes terminal submission audits and compiled PDF from payload hashes to avoid self-reference.
Submission support and venue gate	Traces target-venue decision support, support-packet consistency, and author-gated submission metadata.	Submission package, target venue brief and author-input packet	Supports a source-backed venue decision path; final venue, declarations, archive links, and anonymous-review status remain author input.
Reproducibility package	Documents compute envelope, worker manifests, artifact counts, raw rerun entry points, and availability section.	Experimental Design and Data and Code Availability	Repository-relative package until an archival DOI or anonymous link is supplied.

The submission archive manifest in Table 13 adds a package-level check over the stable payload groups. It hashes source files, tables, figures, raw measurements, summaries, manifests, scripts, trained models, and local tool adapters while excluding terminal submission audits and the compiled PDF from the digest set. This exclusion avoids self-referential hashes because the final PDF contains the manifest table itself; PDF existence and page count are checked separately by the readiness audit.

Table 13: Submission archive manifest. Each row reports a stable payload group, the number of files found, missing expected files, a category digest, and the scope boundary for that group.

Payload group	Files	Missing	Digest	Boundary
Manuscript source	4	0	`1c4fbd7299af`	Compiled PDF is checked by readiness audit to avoid self-referential hashes.
Paper tables	163	0	`86f5407828a3`	Terminal submission audit tables are excluded from the stable payload digest.
Submission figures	28	0	`a9bdb91e8e03`	Includes generated figure assets and plotted source data, not raw benchmark reruns.
Raw measurements	145	0	`a868b9542153`	Raw CSVs are regenerated only by the heavier run scripts, not by the lightweight rebuild.
Derived summaries	360	0	`d034d8579d1d`	Terminal submission/package outputs are excluded so this manifest remains stable.
Run manifests	91	0	`82ac16fc4670`	Submission-level terminal manifests are excluded from this digest group.
Scripts and docs	119	0	`5904ae212489`	Includes reproducible entry points and top-level core implementation modules; raw sweeps still require their individual drivers.
Models	20	0	`4c5c2a1bfc19`	Includes trained local policy artifacts when present; model retraining is not part of the lightweight rebuild.
External adapters	1	0	`de5aa3b78998`	Includes local adapter source files used for external toolchain probes, not vendored tool repositories.
Submission support	11	0	`49b34c13ee40`	Includes public package README, author-input packet, artifact guide, cover-letter, declaration, checklist, reviewer-brief, editor-screening, venue-selection, and structured metadata templates; ignored generated private submission text is excluded.

6 Results

6.1 Traditional functions

Figure 2 and Table 14 summarize the traditional $n \leq 6$ slice. Pareto-Resource-NMCTS has the lowest mean T-count and weighted score among the shown internal baselines, reducing T-count by 72.25% and score by 67.80% relative to direct ANF synthesis. SSHR-H retains the lowest mean CNOT count, so the result is not a CNOT-only dominance claim.

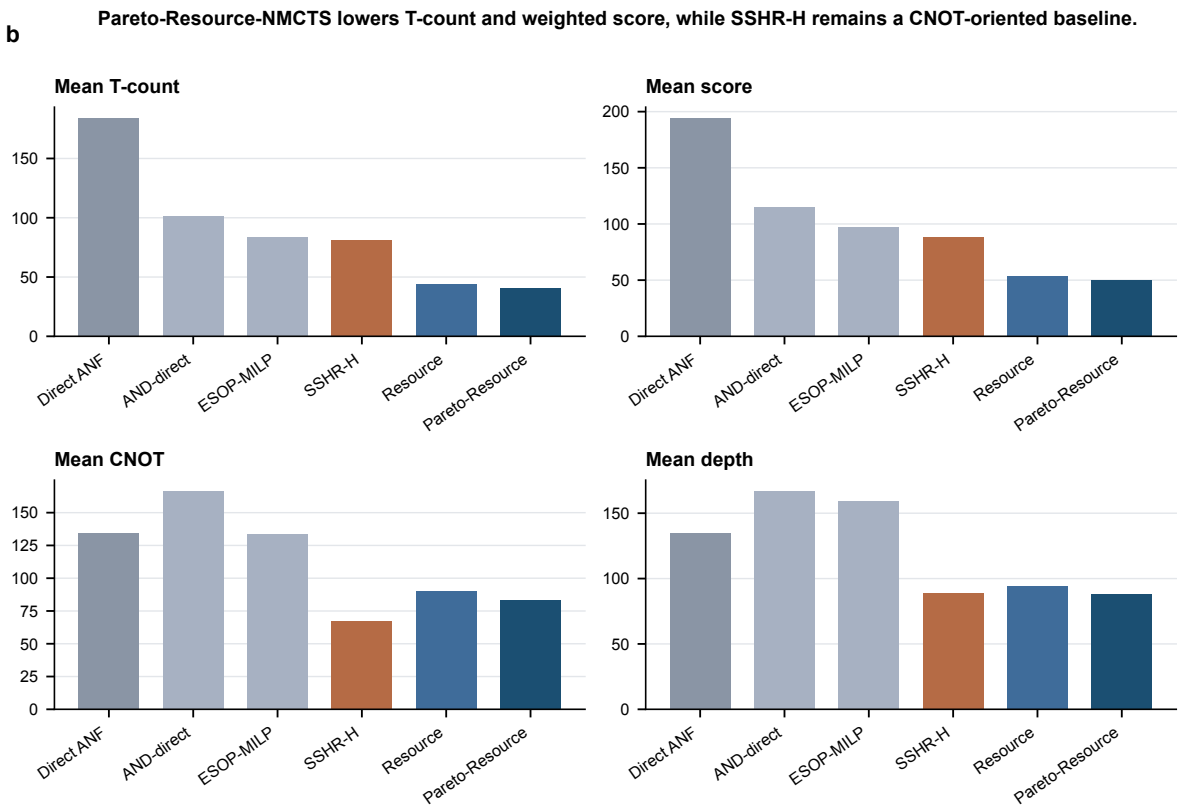

Figure 2: **Mean logical resources on the traditional slice.** Pareto-Resource-NMCTS has the lowest T-count and weighted score; SSHR-H remains a strong CNOT-oriented baseline.

Table 14: Mean logical resources on the $n \leq 6$ traditional slice.

Method	Mean T	Mean CNOT	Mean depth	Mean ancilla	Mean score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	**67.6**	88.7	1.36	88.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	**40.4**	83.0	**88.0**	2.03	**49.6**

6.2 External logic-network and reversible-synthesis probes

Figure 3 summarizes paired score changes against several external or traditional baselines. The ROS-style LUT proxy covers 309 functions and 927 K-sweep rows, all truth-table checked; Pareto-Resource-NMCTS wins 309/309 score comparisons against the best-K proxy. The mockturtle probe uses official headers to resynthesize KLUT networks into XAGs; Pareto-Resource-NMCTS has 166/11/0 score wins on the traditional set and 64/0/0 on the $n = 14$ set.

We further reselect the same verified LUT sweep under stricter line pressure. The minimum-ancilla selector reduces the LUT proxy's peak ancilla by 32.47% relative to the best-score proxy, but increases its own weighted score by 40.67%. Pareto-Resource-NMCTS still obtains 309/0/0 score wins against this minimum-ancilla selector and against two line-weighted selectors. This result is a robustness check for LUT-parameter choice, not a claim that the official ROS SAT garbage-management flow has been reproduced.

Table 15: Line-aware reselection of the verified ROS-style LUT proxy sweep.

Comparison	Metric	Items	W/L/T	Mean Δ
and_pareto_resource_nmcts vs external_ros_lut_min_ancilla	score	309	309/0/0	-85.83%
and_pareto_resource_nmcts vs external_ros_lut_min_ancilla	peak_ancilla	309	301/0/8	-68.21%
and_pareto_resource_nmcts vs external_ros_lut_line_w10	score	309	309/0/0	-84.45%
and_pareto_resource_nmcts vs external_ros_lut_k4_line_cap	score	309	309/0/0	-85.02%
external_ros_lut_min_ancilla vs external_ros_lut_proxy	peak_ancilla	309	197/0/112	-32.47%
external_ros_lut_min_ancilla vs external_ros_lut_proxy	score	309	0/197/112	+40.67%

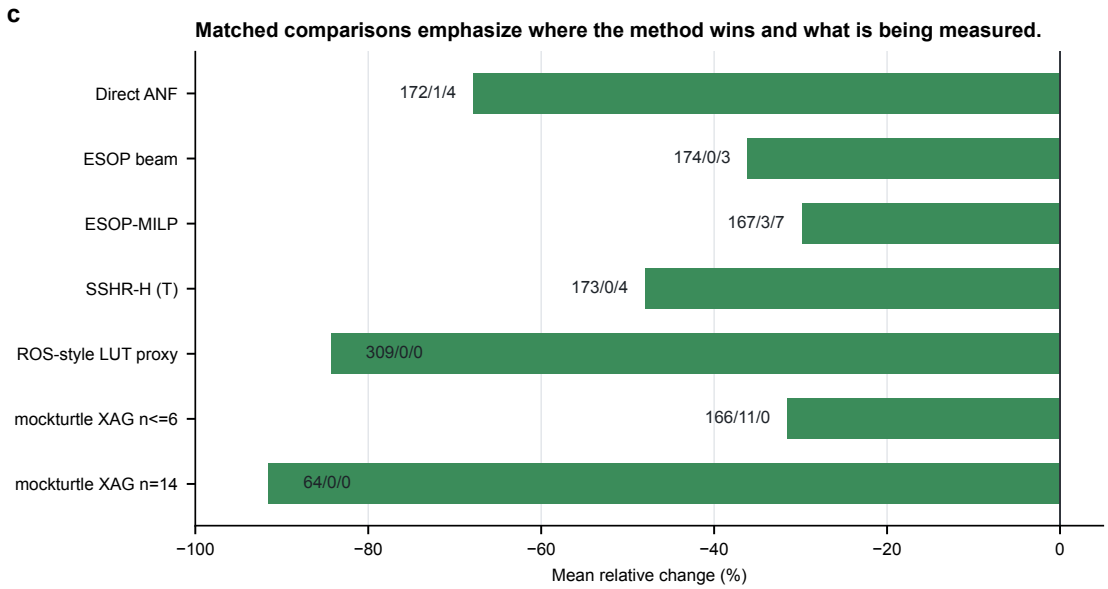

Figure 3: **Paired baseline comparisons.** Negative values favor the target method. Labels show win/loss/tie counts.

The CirKit probe converts the same BLIF benchmarks to AIGER with ABC, applies CirKit AIG rewriting and multiplicative-complexity analysis, writes Verilog, and verifies the Verilog readback through ABC. It produces 177/177 correct traditional rows and 64/64 correct $n = 14$ rows. Pareto-Resource-NMCTS wins all score comparisons but loses depth on most rows, making CirKit the strongest depth-oriented external probe in the current manuscript.

Table 16: CirKit 3 AIG/multiplicative-complexity probe on $n \leq 6$ functions.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	177	177/0/0	-60.84%
and_pareto_resource_nmcts	177	177/0/0	-62.34%
and_fprm_polarity_archive	177	177/0/0	-58.96%
and_direct_anf	177	124/53/0	-16.74%
direct_anf	177	46/131/0	+35.23%
and_esop_milp	177	150/27/0	-39.41%
sshr_h	177	164/13/0	-32.83%

Table 17: CirKit 3 AIG/multiplicative-complexity probe on the $n = 14$ set.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	64	64/0/0	-94.42%
and_profile_resource_nmcts	64	64/0/0	-94.42%
and_pareto_resource_nmcts	64	64/0/0	-94.46%
and_fprm_linear_pair	64	64/0/0	-94.27%
and_fprm_root_beam	64	64/0/0	-94.11%
and_direct_anf	64	64/0/0	-91.76%
direct_anf	64	64/0/0	-86.22%
and_fprm_greedy	64	64/0/0	-94.08%
and_affine_greedy	64	64/0/0	-94.08%

The RevKit CLI probe supplies a closer reversible-synthesis comparison. Each function is embedded as the exact oracle permutation $(x, y) \mapsto (x, y \oplus f(x))$, and legacy RevKit reads the SPEC permutation before running TBS, DBS, and RMS synthesis. All 531 direct flow rows return. The per-function best-score RevKit portfolio is strong on line count, but Pareto-Resource-NMCTS retains 173/0/4 score wins and a 67.28% mean score reduction.

Table 18: Legacy RevKit CLI reversible-synthesis portfolio on $n \leq 6$ functions. Each function uses the best-score result among TBS, DBS, and RMS.

Target vs RevKit CLI best-score	Items	W/L/T	Mean Δ
and_resource_nmcts	177	173/0/4	-66.32%
and_pareto_resource_nmcts	177	173/0/4	-67.28%
and_fprm_polarity_archive	177	173/0/4	-64.24%
and_direct_anf	177	167/6/4	-32.79%
direct_anf	177	87/86/4	+5.78%
and_esop_milp	177	172/1/4	-51.50%
sshr_h	177	170/7/0	+83.84%

6.3 Paired statistical evidence

The headline reductions above are paired by function or item, but mean relative improvements alone can hide skewed instances. We therefore recompute the central score comparisons directly from usable raw rows and apply an exact two-sided sign test that ignores ties. Table 19 shows that the main traditional, external-toolchain, and high-dimensional score claims have negative median relative changes as well as strongly one-sided win counts. The high-dimensional root-beam and fast-pair margins are smaller in magnitude than the external-probe margins, but they remain consistent across paired rows.

Table 19: Paired statistical evidence for score comparisons. Rows are recomputed from correct, non-skipped raw CSV entries matched by item name. Negative mean and median changes favor the target method; $\log_{10} p$ is from a two-sided exact sign test over wins and losses, ignoring ties.

Comparison	Pairs	W/L/T	Mean Δ	Median Δ	$\log_{10} p$
$n \leq 6$ Pareto vs direct ANF	177	172/1/4	−67.80%	−71.37%	-49.54
$n \leq 6$ Pareto vs ESOP beam	177	174/0/3	−36.09%	−32.56%	-52.08
$n \leq 6$ Pareto vs ESOP-MILP	177	167/3/7	−29.84%	−23.69%	-44.96
$n \leq 6$ Pareto vs SSHR-H	177	173/4/0	−41.06%	−43.05%	-45.37
$n \leq 6$ Pareto vs SSHR-I CNOT	177	173/4/0	−31.89%	−33.91%	-45.37
$n \leq 6$ Pareto vs ABC-XAG	177	177/0/0	−65.58%	−65.25%	-52.98
ROS-style LUT best-K	309	309/0/0	−84.27%	−81.38%	-92.72
mockturtle XAG $n \leq 6$	177	166/11/0	−31.50%	−32.43%	-35.96
mockturtle XAG $n = 14$	64	64/0/0	−91.49%	−95.50%	-18.96
CirKit AIG/MC $n \leq 6$	177	177/0/0	−62.34%	−62.35%	-52.98
CirKit AIG/MC $n = 14$	64	64/0/0	−94.46%	−96.10%	-18.96
RevKit CLI exact oracle	177	173/0/4	−67.28%	−70.18%	-51.78
$n = 14$ Pareto vs root beam	64	60/0/4	−6.31%	−4.68%	-17.76
$n = 16$ Resource vs root beam	24	23/0/1	−4.36%	−3.36%	-6.62
$n = 18$ Resource vs fast pair	12	12/0/0	−3.55%	−1.79%	-3.31

To avoid treating each mean effect as a single point estimate, we also compute nonparametric bootstrap intervals on the same matched rows. This uncertainty check does not introduce new measurements; it asks whether the observed effect sizes remain separated from zero when functions or items are resampled with replacement. Table 20 shows that the main small-function and external-toolchain comparisons remain negative with narrow intervals, while the high-dimensional root-beam and fast-pair comparisons keep smaller but still negative intervals. The table therefore strengthens the paired evidence without converting a weighted-score result into a universal resource-dominance claim.

Table 20: Bootstrap uncertainty intervals for paired score-effect estimates. Rows reuse the matched pairs from Table 19. Intervals are percentile bootstrap intervals over relative score changes; negative values favor the target method.

Comparison	Pairs	Mean Δ [95% CI]	Median Δ [95% CI]	Item p10/p90
$n \leq 6$ Pareto vs direct ANF	177	−67.80% [−69.91, −65.42]	−71.37% [−73.09, −69.62]	−78.88/−54.46
$n \leq 6$ Pareto vs ESOP beam	177	−36.09% [−38.67, −33.69]	−32.56% [−35.11, −29.81]	−55.01/−19.65
$n \leq 6$ Pareto vs ESOP-MILP	177	−29.84% [−32.70, −27.04]	−23.69% [−26.68, −21.47]	−62.94/−10.26
$n \leq 6$ Pareto vs SSHR-H	177	−41.06% [−43.38, −38.77]	−43.05% [−44.08, −39.82]	−55.50/−22.11
$n \leq 6$ Pareto vs SSHR-I CNOT	177	−31.89% [−33.80, −29.90]	−33.91% [−36.68, −33.23]	−46.91/−14.30
$n \leq 6$ Pareto vs ABC-XAG	177	−65.58% [−67.10, −64.11]	−65.25% [−66.60, −63.34]	−76.73/−54.77
ROS-style LUT best-K	309	−84.27% [−85.73, −82.79]	−81.38% [−83.16, −79.32]	−99.62/−68.49
mockturtle XAG $n \leq 6$	177	−31.50% [−34.31, −28.62]	−32.43% [−33.58, −29.70]	−49.55/−9.96
mockturtle XAG $n = 14$	64	−91.49% [−94.09, −88.33]	−95.50% [−97.12, −93.60]	−98.93/−83.64
CirKit AIG/MC $n \leq 6$	177	−62.34% [−64.10, −60.59]	−62.35% [−64.66, −61.06]	−76.91/−47.69
CirKit AIG/MC $n = 14$	64	−94.46% [−95.72, −93.16]	−96.10% [−97.33, −94.71]	−98.96/−87.43
RevKit CLI exact oracle	177	−67.28% [−69.11, −65.23]	−70.18% [−71.53, −68.21]	−78.57/−57.54
$n = 14$ Pareto vs root beam	64	−6.31% [−7.89, −4.95]	−4.68% [−5.21, −3.76]	−13.44/−2.05
$n = 16$ Resource vs root beam	24	−4.36% [−6.61, −2.89]	−3.36% [−4.22, −2.46]	−6.53/−1.55
$n = 18$ Resource vs fast pair	12	−3.55% [−5.66, −1.81]	−1.79% [−4.50, −1.29]	−8.59/−1.24

6.4 Raw multi-resource dominance

Because Eq. (3) is only a ranking objective, we also test dominance without using the weighted score. A method is said to dominate another row only if it is no worse in T-count, CNOT count, logical depth, and peak ancilla, and is strictly better in at least one of them. Table 21 shows the resulting four-resource dominance counts for Pareto-Resource-NMCTS on the traditional slice. The method strongly dominates ESOP beam and many ESOP-MILP rows, but most comparisons against SSHR, CirKit, mockturtle, and RevKit are incomparable. This confirms

that those baselines occupy real CNOT, depth, or line-count tradeoff points even when their weighted scores are worse.

Table 21: Raw four-resource dominance of Pareto-Resource-NMCTS on $n \leq 6$ functions. Dominance uses only T-count, CNOT count, logical depth, and peak ancilla; weighted score is not part of the predicate.

Baseline	Pairs	Pareto dom.	Base dom.	Incomp.	Equal
Direct ANF	177	69	1	103	4
ESOP beam	177	165	0	9	3
ESOP-MILP	177	123	2	45	7
SSHR-H	177	33	4	140	0
SSHR-I CNOT	177	9	3	165	0
ABC-XAG	177	33	0	144	0
mockturtle XAG	177	11	0	166	0
CirKit AIG/MC	177	21	0	156	0
RevKit CLI exact oracle	177	3	0	170	4

The same predicate can be applied to the whole method pool. Table 22 counts how often each method remains non-dominated by any other listed method on the same function. Pareto-Resource-NMCTS is non-dominated on 170/177 functions, while SSHR-I, mockturtle, and RevKit remain frequently non-dominated because they keep distinct resource tradeoffs.

Table 22: Non-dominated counts in the 12-method $n \leq 6$ pool.

Method	Rows	Non-dominated	Dominated
Pareto-Resource-NMCTS	177	170	7
SSHR-I CNOT	177	162	15
mockturtle XAG	177	159	18
RevKit CLI exact oracle	177	141	36
Resource-NMCTS	177	124	53
CirKit AIG/MC	177	56	121
SSHR-H	177	46	131
ESOP-MILP	177	44	133
ABC-XAG	177	15	162
Direct ANF	177	7	170
ESOP beam	177	6	171
AND-direct ANF	177	4	173

6.5 Search contribution and ablation

Table 23 consolidates the contribution analysis. The largest pre-portfolio gain comes from affine and Boolean-ring structural search. The learned prior gives smaller but non-degrading quality gains on the traditional slice, while the frontier policies mainly control the quality/time frontier in high-dimensional term-set experiments. Table 27 isolates the high-dimensional Boolean-ring and Boolean-screen branch. This table is deliberately mixed: it shows quality-seeking structural guards, a screen gate that preserves the selected resource vector while saving time, and a negative control where a fast screen is not resource-competitive. This prevents the structural-search claim from being reduced to a neural-prior claim or overstated as a universal screen improvement. Table 28 then compresses the expensive depth-frontier itself. On the listed scale and truth-bridge slices, evaluating only depth 2 and depth 4 exactly matches the full depth-2/3/4 frontier while removing 24.94–28.88% of frontier evaluation time. Table 29 adds a learned gate on top of this sparse frontier. Trained on generated $n = 16, 20, 24$ term sets and applied unchanged to held-out $n = 28, 40$ term sets, it runs depth 4 on 32/48 test pairs, records zero false skips, exactly matches the sparse-frontier score, and reduces sparse-frontier evaluation time by 17.39%. The deterministic sparse frontier remains the quality reference; the learned component controls search budget after the depth-2 screen. Table 30 then audits the same trained gate without

retraining on three independent seeds over $n = 24, 28, 32, 40$. Across 144 additional pairs it records zero false skips, matches the deterministic sparse frontier on every row, and saves 13.43% of sparse-frontier evaluation time. Figure 4 then sweeps the deployment threshold on the same audit. The selected threshold lies inside a zero-false-skip plateau; the best zero-false-skip point in the sweep saves 14.92%, while allowing one false skip saves 15.49% with a 0.01% mean score gap. Table 31 adds the earlier staged teacher/guard. Its depth-4 trigger is selected only on the large-policy validation split; on the independent $n = 24, 28, 32, 40$ scale rows it remains within 0.04% of all-depth score while reducing staged planning time by 25.43%.

Table 23: Matched contribution decomposition. Negative mean score changes favor the first method in each comparison.

Component	Dataset	Pairs	Score W/L/T	Mean Δ score
Affine greedy vs fixed-coordinate MCTS	ablation_affine	321	165/88/68	−12.12%
Neural refine over affine greedy	ablation_affine	322	65/0/257	−1.08%
Guarded Affine-NMCTS over no-guard	ablation_affine	322	88/0/234	−1.74%
Learned prior for Resource-NMCTS	traditional_resource	177	39/0/138	−1.10%
Pareto archive over Resource-NMCTS	traditional_resource	177	68/0/109	−3.26%
No-MCTS portfolio over heuristic-only	trad. ablation	177	54/0/123	−2.51%
Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	54/0/123	−1.44%
Pareto Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	106/0/71	−4.69%
Highdim no-MCTS portfolio over root beam	n14 ablation	16	14/0/2	−6.25%
Highdim no-MCTS portfolio over linear-pair	n14 ablation	16	14/0/2	−3.08%
Linear-pair guard vs root beam at n=14	highdim_resource	64	60/0/4	−3.00%
Recursive linear-pair guard vs root beam at n=15	n15 scale	32	30/0/2	−5.28%
Recursive linear-pair guard vs shallow linear-pair at n=16	n16 scale	24	23/0/1	−2.54%
Recursive linear-pair guard vs root beam at n=16	n16 scale	24	23/0/1	−4.31%
Baseline-preserving AI guard vs deterministic recursive guard at n=16	n16 scale	24	6/0/18	−0.05%
Fast linear-pair guard vs root beam at n=18	n18 stress	12	6/0/6	−1.91%
Resource-NMCTS vs fast linear-pair at n=18	n18 stress	12	12/0/0	−3.55%

Table 24 restates the search-control comparisons in the form used for claim interpretation. The bit-flip branch is compared against heuristic-only, beam-only, strengthened no-MCTS portfolio, MCTS, Pareto archive, and learned-prior/no-prior controls. Table 25 adds a same-budget random-prior control for the bit-flip neural scorer. The learned prior changes only action ranking, keeps the same candidate legality and semantic checks, and is compared with eight deterministic random-prior repeats per function. The effect is intentionally reported as small: learned scores are non-degrading and beat the random-prior mean, but the largest resource gains still depend on the algebraic action space and baseline-preserving guards.

Table 24: Search-control baseline audit. Rows state which search/control baseline is being tested and which stronger claim remains outside the evidence for that comparison.

Layer	Comparison	Pairs	Score W/L/T	Mean Δ score	Claim boundary
search-space baseline	Affine greedy vs fixed-coordinate MCTS	321	165/88/68	-12.12%	This is not a neural-only effect.
deterministic search controls	No-MCTS portfolio over heuristic-only	177	54/0/123	-2.51%	This row does not isolate MCTS.
deterministic search controls	No-MCTS portfolio over beam-only	177	106/1/70	-8.36%	Beam is a child/search baseline, not the full method.
MCTS over deterministic portfolio	Resource-NMCTS over no-MCTS portfolio	177	54/0/123	-1.44%	The magnitude is incremental, not the whole resource drop.
Pareto search control	Pareto Resource-NMCTS over no-MCTS portfolio	177	106/0/71	-4.69%	Ancilla tradeoffs still need separate reporting.
learned prior	Learned prior for Resource-NMCTS	177	39/0/138	-1.10%	It is not a runtime claim and should not be promoted as the main source of improvement.
bit-flip random control	Learned bit-flip prior vs same-budget random-prior mean	177	17/8/152	-0.15%	The margin is small and runtime-negative; this is not a claim that neural ranking explains the full gain.
frontier random-depth control	Large frontier policy vs same-candidate random depth	102	55/3/38; 5/0/1	-1.12%	The control is quality-positive but runtime-negative against random depth; it is not a speed, hardware-scheduling, or global-optimality claim.
high-dimensional deterministic control	Highdim no-MCTS portfolio over root beam	16	14/0/2	-6.25%	This is symbolic/high-dimensional evidence, not exhaustive truth-table optimality.
phase random control	Diverse phase shortlist top-512 vs same-budget random mean	38	17/0/21	-0.012%	This is a phase/Rz proxy control, not a bit-flip random baseline or rotation synthesis.

Table 25: Same-budget bit-flip random-prior control. W/L/T compares the learned prior with the per-function mean over eight deterministic random-prior repeats; negative score changes favor the learned prior.

Method	Pairs	Repeats	W/L/T	Best-random wins	Mean score change	Boundary
Affine-Resource-NMCTS	177	8	20/12/145	165/177	−0.12%	same-budget random-prior scorer; lower score favors learned prior
Resource-NMCTS	177	8	17/8/152	169/177	−0.15%	same-budget random-prior scorer; lower score favors learned prior
Pareto-Resource-NMCTS	177	8	5/5/167	172/177	−0.03%	same-budget random-prior scorer; lower score favors learned prior

Table 26 applies the same random-control idea to the large Boolean-ring frontier policy. The candidate set is fixed to the already verified depth-2/3/4 screens, and only the depth-selection policy is replaced by eight deterministic random choices. The learned policy beats the random-depth mean on held-out, independent scale, and $n = 23$ truth-bridge rows, but it spends more planning time because it selects deeper screens more often. We therefore use this control as

quality-oriented evidence for learned budget allocation, not as a runtime claim.

Table 26: Same-candidate random-depth control for the Boolean-ring frontier policy. W/L/T compares the learned large frontier policy with the per-function mean over eight deterministic random depth choices among the already verified depth-2/3/4 candidates.

Source	Scope	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Boundary
held-out frontier policy	test n=28,40	48	24/1/23	-0.78%	+59.14%	beats 8/8 random seed means; quality-oriented, not a runtime claim
scale generalization	n=24,28,32,40	96	55/3/38	-1.12%	+58.78%	beats 8/8 random seed means; quality-oriented, not a runtime claim
truth-table bridge	n=23	6	5/0/1	-0.89%	+71.00%	beats 8/8 random seed means; quality-oriented, not a runtime claim

Table 27: Boolean-ring structural evidence in high-dimensional slices. Negative mean changes favor the first method in each comparison. The $n = 18$ row is kept as a speed-only negative control rather than promoted as a quality result.

Slice	Comparison	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Interpretation
$n = 14$	Boolean guard vs pairwise-wide	12	9/0/3	-0.82%	+32.80%	quality gain; slower exhaustive guard
$n = 16$	Boolean guard vs pairwise-wide	24	14/0/10	-0.34%	+22.80%	quality gain; slower guard
$n = 16$	Boolean guard vs deterministic deep	24	18/0/6	-0.52%	+356.64%	stronger quality branch; expensive
$n = 16$	Boolean-linear deep vs pairwise-wide	24	14/6/4	-0.22%	-72.31%	fast structural variant with small quality gain
$n = 20$	Resource update vs Boolean screen	6	5/0/1	-4.52%	+453.05%	quality frontier; much slower
$n = 20$	Screen gate vs full Resource	6	0/0/6	+0.00%	-75.58%	safe skip: same resources, less planning time
$n = 20$	Recursive Boolean screen vs old Resource	6	5/0/1	-7.47%	-7.32%	large-n positive screen; ancilla tradeoff
$n = 18$	Recursive Boolean screen vs old Resource	12	0/11/1	+36.94%	-97.90%	speed-only negative control; not promoted

Table 28: Sparse depth-frontier audit. The sparse controller evaluates depth-2 and depth-4 Boolean-ring screens, skips depth 3, and is compared with the full measured depth-2/3/4 frontier in the same raw files.

Source	n scope	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Depth-4 vs depth-3
scale-frontier	20,28,40	72	0/0/72	+0.00%	-27.57%	44/0/28
scale-generalization	24,28,32,40	96	0/0/96	+0.00%	-28.17%	52/0/44
truth-bridge-n23	23	6	0/0/6	+0.00%	-24.94%	5/0/1
truth-bridge-n24	24	6	0/0/6	+0.00%	-28.88%	3/0/3
truth-bridge-n25	25	6	0/0/6	+0.00%	-27.06%	4/0/2

Table 29: Learned sparse depth-4 gate. The gate predicts whether the depth-4 Boolean-ring screen should run after the depth-2 sparse-frontier state has been evaluated. The threshold is selected on validation data with a conservative false-skip guard and is applied unchanged to held-out $n = 28, 40$ test rows.

Split	Pairs	Run d4	False skips	Score W/L/T vs sparse	Mean Δ score	Mean Δ time	Score W/L/T vs depth-2
train	96	78	0	0/0/96	+0.00%	-10.17%	75/0/21
valid	48	38	0	0/0/48	+0.00%	-12.01%	34/0/14
test	48	32	0	0/0/48	+0.00%	-17.39%	23/0/25

Table 30: Independent-seed sparse depth-4 gate audit. The trained gate and its validation-selected threshold are reused without retraining on three new seeds. The table reports aggregate and per-n comparisons against the deterministic sparse depth-2/4 frontier.

Scope	Pairs	Run d4	False skips	Score W/L/T vs sparse	Mean Δ score	Mean Δ time	Score W/L/T vs depth-2
all	144	106	0	0/0/144	+0.00%	-13.43%	94/0/50
n=24	36	29	0	0/0/36	+0.00%	-9.88%	27/0/9
n=28	36	27	0	0/0/36	+0.00%	-12.98%	24/0/12
n=32	36	23	0	0/0/36	+0.00%	-18.19%	21/0/15
n=40	36	27	0	0/0/36	+0.00%	-12.66%	22/0/14

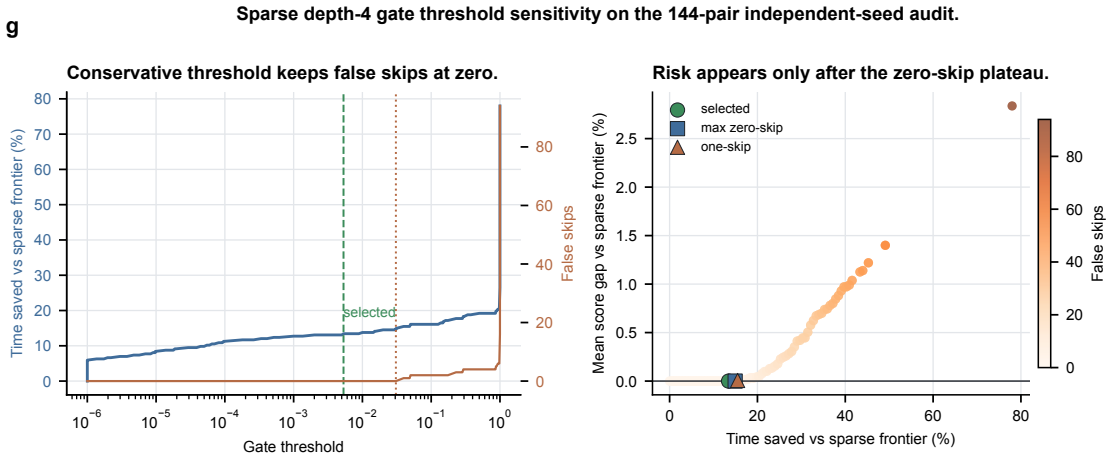

Figure 4: **Sparse depth-4 gate threshold sensitivity.** The sweep reuses the trained gate on the 144-pair independent-seed audit. The validation-selected threshold lies inside a zero-false-skip plateau, showing that the gate acts as a calibrated search-budget controller rather than as an unguarded depth-4 skip.

Table 31: Stage-gated depth-frontier controller. The depth-4 trigger is chosen on validation data and applied unchanged to scale and truth-bridge rows.

Source	Baseline	Pairs	Score W/L/T	Mean Δ score	Mean Δ time	Run depth-4
scale_generalization	adaptive_all_depth	96	0/4/92	+0.04%	-25.43%	52
scale_generalization	screen_depth2	96	58/0/38	-2.40%	+893.25%	52
scale_generalization	depth_frontier_policy	96	5/3/88	-0.06%	+101.10%	52
truth_bridge_n23	adaptive_all_depth	6	0/0/6	+0.00%	-11.51%	5
truth_bridge_n23	screen_depth2	6	5/0/1	-2.47%	+1322.09%	5
truth_bridge_n23	depth_frontier_policy	6	1/0/5	-0.12%	+94.20%	5

Table 32 reports a compact schedule-proxy view of the same high-dimensional frontier family. The metrics are computed from the emitted logical X/CNOT/MCT circuits before hardware mapping. Lower values are better. Relative to the cheap depth-2 screen, the frontier policy reduces score and T-depth proxy but increases explicit auxiliary lifetime area. Relative to the full all-depth frontier, it accepts a small score/T-depth gap while reducing auxiliary lifetime area. This is the intended resource-constrained interpretation: the learned controller moves along a quality–search-budget– auxiliary-lifetime frontier rather than claiming hardware-scheduled depth dominance.

Table 32: Logic-level schedule-proxy audit. Entries show win/loss/tie counts and mean relative change for the first method versus the second. The metrics are backend-relevant logical proxies only; they are not hardware mapping, routing, or native-gate scheduling results.

Evidence slice	Comparison	Score	T-depth proxy	Aux. lifetime area	Boundary
$n = 24, 28, 32, 40$ scale	learned frontier vs cheap depth-2	40/0/56; -1.85%	40/0/56; -1.85%	0/40/56; +20.09%	quality gain with more auxiliary lifetime
$n = 24, 28, 32, 40$ scale	learned frontier vs all-depth ceiling	0/19/77; +0.61%	0/19/77; +0.55%	19/0/77; -5.42%	near-ceiling quality with lower lifetime
$n = 21, 22$ bridge	complete-truth bridge vs cheap depth-2	8/0/4; -3.50%	8/0/4; -3.32%	0/8/4; +32.93%	truth-checked quality gain with lifetime cost
$n = 21, 22$ bridge	complete-truth bridge vs all-depth ceiling	0/2/10; +0.32%	0/2/10; +0.32%	2/0/10; -4.01%	small score gap with lower lifetime
$n = 23$ bridge	larger truth bridge vs cheap depth-2	4/0/2; -1.88%	4/0/2; -1.69%	0/4/2; +29.49%	quality gain with lifetime cost
$n = 23$ bridge	larger truth bridge vs all-depth ceiling	0/1/5; +0.61%	0/1/5; +0.52%	1/0/5; -5.09%	small score gap with lower lifetime
$n = 24, 28, 32, 40$ scale	all-depth ceiling vs cheap depth-2	58/0/38; -2.44%	58/0/38; -2.37%	0/58/38; +27.81%	quality ceiling increases lifetime
$n = 23$ bridge	bridge all-depth ceiling vs cheap depth-2	5/0/1; -2.47%	5/0/1; -2.20%	0/5/1; +36.83%	truth-checked ceiling increases lifetime

Table 33 separates promoted learned-control components from limited diagnostics. The frontier policies, sparse depth-4 gate, and phase shortlist provide the paper-facing learned-control evidence. In contrast, the current Boolean-neural guard and root-action neural ranker are kept as bounded diagnostics: they show small quality or evaluation-cost signals, but not enough quality/runtime advantage to carry the main claim.

Table 33: Learned-control evidence audit. Rows marked as limited are reported to bound the role of AI rather than promoted as headline improvements.

Component	Scope	Quality evidence	Cost evidence	Paper role
Depth-frontier policy	held-out $n = 28, 40$; 48 rows	vs oracle frontier 0/3/45, +0.04%	-51.30% time vs all-depth frontier	promoted quality/time selector
Frontier random-depth control	held-out/scale/$n = 23$; 8 random-depth repeats	scale 55/3/38, -1.12%; $n = 23$ 5/0/1, -0.89%; seed means beaten 8/8	+58.78% scale planning time vs random-depth mean	quality-oriented budget allocation; not runtime claim
Stage-gated frontier	independent $n = 24, 28, 32, 40$; 96 rows	vs all-depth 0/4/92, +0.04%	-25.43% staged planning time	promoted validation-calibrated guard
Sparse depth-4 gate	multi-seed $n = 24, 28, 32, 40$; 144 pairs	vs sparse frontier 0/0/144, +0.00%; false skips 0	-13.43% time vs sparse frontier	promoted budget gate after depth-2
Rank-diverse phase shortlist	held-out $n = 6$ phase search; 38 rows	vs budget-32 17/0/21, -2.48%; vs wide-128 0/7/31, +0.00%	512/8192 exact forms per function	promoted phase-search pruning
Bit-flip learned prior	177 n¡=6 functions; 8 random-prior repeats	vs random mean 17/8/152, -0.15%; seed means beaten 8/8	+48.05% runtime vs random-prior mean	limited quality signal, not runtime claim
Boolean neural guard	$n = 16$ high-dimensional guard; 24 rows	vs deterministic 4/0/20, -0.12%	+94.49% runtime	limited quality guard, not runtime claim
Root-action neural ranker	$n = 14$ root-action diagnostic; 10 rows	vs beam4 +0.03%; oracle24 headroom -0.12%	-98.06% ranking time vs beam4 eval	not promoted; future root-ranker target

Figure 5 visualizes the same conservative separation. Promoted controllers either preserve the reference score within a small tolerance or improve it, while reducing the measured planning time or exact-evaluation budget. The two limited diagnostics fall outside this quality–effort pattern and are therefore kept out of the headline claim.

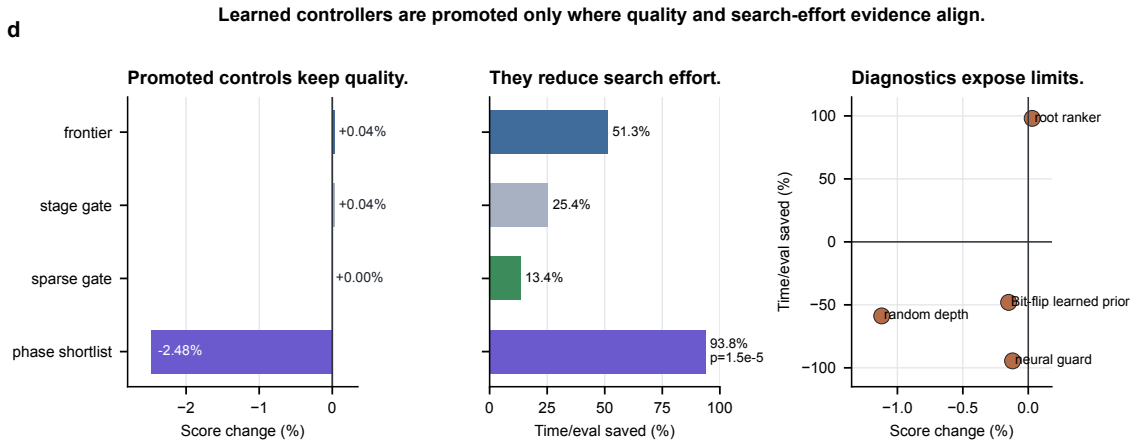

Figure 5: **Learned-control summary.** Promoted learned controllers are reported only where quality and search-effort evidence align. Positive time/evaluation saving means fewer frontier evaluations, less staged planning time, or fewer exact-scored phase candidates; negative score change favors the learned controller.

6.6 Score-weight robustness

The active score in Eq. (3) is intentionally a logical-resource ranking objective rather than a physical cost model. To check that the main comparisons are not artifacts of one coefficient choice, we recompute scores post hoc on verified rows under T-only, CNOT/depth-heavy, and

ancilla-tight profiles. Table 34 shows that the small-function advantages over ESOP and SSHR-H and the high-dimensional advantages over root-beam or fast-pair guards survive these alternative profiles, while the CNOT/depth and ancilla-tight profiles expose the expected narrower margins.

Table 34: Post-hoc robustness to alternative logical-resource weights. Each cell reports score win/loss/tie counts and mean relative score change; negative values favor Resource-NMCTS or Pareto-Resource-NMCTS.

Comparison	Paper score	T-only	CNOT-depth	Ancilla-tight
$n \leq 6$ traditional: ESOP cube beam	174/0/3, -36.09%	174/0/3, -38.95%	174/0/3, -32.08%	174/0/3, -32.75%
$n \leq 6$ traditional: ESOP-MILP	167/3/7, -29.84%	166/0/11, -32.77%	165/4/8, -25.44%	167/3/7, -26.68%
$n \leq 6$ traditional: SSHR-H	173/4/0, -41.06%	173/0/4, -47.93%	168/9/0, -27.87%	172/5/0, -31.95%
$n = 14$ random ANF: FPRM root beam	60/0/4, -6.31%	60/0/4, -7.65%	60/0/4, -5.35%	56/4/4, -3.28%
$n = 16$ random ANF: FPRM root beam	23/0/1, -4.36%	23/0/1, -4.70%	23/0/1, -4.28%	23/0/1, -3.43%
$n = 18$ random ANF: FPRM root beam	12/0/0, -5.29%	12/0/0, -6.19%	12/0/0, -4.54%	11/1/0, -3.33%
$n = 18$ random ANF: fast pair guard	12/0/0, -3.55%	12/0/0, -3.75%	12/0/0, -3.23%	12/0/0, -3.33%

6.7 Phase/Rz search

RevKit Python `oracle_synth` returns on all 177 traditional functions, but 171 rows contain non-Clifford R_z rotations, with 9242 such rotations in total. We therefore treat it as a phase/Rz lower-bound boundary rather than as an exact Clifford+T T-count comparison. The internal phase-parity emitter, fixed-polarity FPRM search, and affine FPRM phase search turn this boundary into a verifiable search branch.

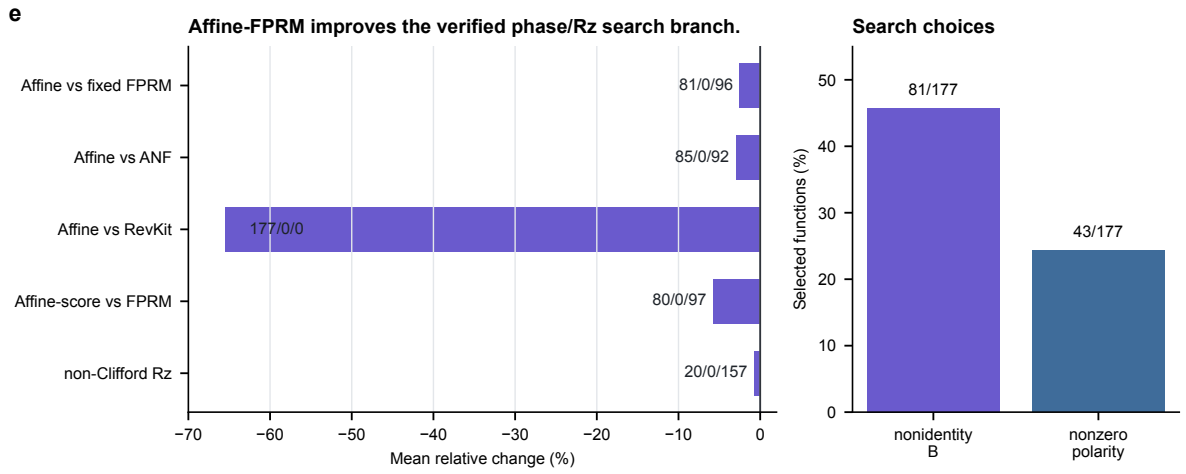

Figure 6: **Affine-FPRM phase search.** Affine preconditioning improves the phase-parity proxy over fixed-polarity FPRM and ANF baselines, while remaining a logical phase/Rz proxy rather than an approximate-rotation sequence.

Table 35: Affine-FPRM phase-parity search comparisons.

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM-$T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM-$T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM-$T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford Rz vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

Increasing the affine transform budget from 32 to 128 yields additional but bounded gains: under the $T/R_z = 30$ proxy, the wide search gives 43/0/134 wins/losses/ties over budget 32 and reduces mean synthesis score by 0.60%. A rank-trained policy and a deterministic diversity reranker then compress this search. On the held-out $n = 6$ split, the diverse top-512 policy exact-scores only 512 of 8192 affine forms per function, keeps the budget-32 comparison at 17/0/21 with a 2.48% mean reduction, and matches the wide-128 mean within 0.01%. Table 38 then uses the eight same-budget random repeats as a control. The independent unit is the held-out function: random repeats are averaged per function before the sign test. The strongest row is the diverse top-512 policy, which is 17/0/21 against the per-function random mean, has sign-test $p = 1.53 \times 10^{-5}$, and beats all eight random seed means. The absolute margins are small because the affine candidate space is dense, so this supports learned pruned phase-search feasibility rather than a claim that the current scorer solves phase/Rz synthesis globally.

Table 36: Affine-FPRM phase-search budget expansion.

Metric	Pairs	W/L/T	Mean Δ
score target	177	38/0/139	−1.59%
score+1/Rz target	177	40/0/137	−0.93%
$T/R_z = 30$ target	177	43/0/134	−0.60%
non-Clifford Rz	177	3/0/174	−0.29%
total Rz	177	35/2/140	−2.39%
CNOT	177	37/2/138	−1.74%
depth	177	41/2/134	−1.93%

Table 37: Rank-trained and diversity-reranked phase candidate pruning on held-out $n = 6$ functions.

Comparison	Items	W/L/T	Mean Δ
Policy top-512 vs budget-32	38	17/0/21	-2.40%
Policy top-512 vs wide-128	38	0/9/29	+0.13%
Diverse policy top-512 vs budget-32	38	17/0/21	-2.48%
Diverse policy top-512 vs wide-128	38	0/7/31	+0.00%
Policy top-64 vs random top-64	38	10/7/21	+1.37%
Diverse policy top-64 vs random top-64	38	10/7/21	+0.56%
Policy top-128 vs random top-128	38	7/6/25	+0.81%
Diverse policy top-128 vs random top-128	38	8/4/26	+0.03%
Policy top-512 total Rz vs budget-32	38	15/0/23	-4.57%
Policy top-512 CNOT vs budget-32	38	16/1/21	-2.80%
Policy top-512 depth vs budget-32	38	16/1/21	-3.29%

Table 38: Same-budget random-repeat control for the learned phase shortlist. Each row compares a learned top-k shortlist with the mean of eight random top-k shortlists on each held-out $n = 6$ function before applying a paired sign test. Negative Δ mean favors the learned shortlist.

Policy	Top-k	W/L/T	Sign p	Mean target	Mean random	Δ mean
Rank	64	16/3/19	0.0044	1482.48	1482.80	-0.021%
Diverse	64	16/3/19	0.0044	1482.40	1482.80	-0.027%
Rank	128	14/5/19	0.0636	1482.29	1482.52	-0.015%
Diverse	128	14/5/19	0.0636	1482.15	1482.52	-0.025%
Rank	256	12/6/20	0.2379	1482.14	1482.32	-0.012%
Diverse	256	15/3/20	0.0075	1482.06	1482.32	-0.018%
Rank	512	14/3/21	0.0127	1482.03	1482.13	-0.007%
Diverse	512	17/0/21	1.53×10^{-5}	1481.95	1482.13	-0.012%

6.8 High-dimensional verification

Figure 7 summarizes the current verification envelope. The manuscript-level evidence includes 5640/5640 item-level symbolic runs for $n = 20$–40, 1512/1512 width-probe symbolic runs, 531/531 exact phase-search rows, 7611/7611 phase-policy pruning rows, and 400/400 complete truth table bridge method rows. The complete bridge spans $n = 21$–25 and checks the oracle relation, the expanded ANF plan, and the emitted-circuit symbolic semantics. Table 39 makes the large-scale evidence explicit by separating generated functions or settings from method rows and by reporting representative resource means only for the paper-facing method in each slice.

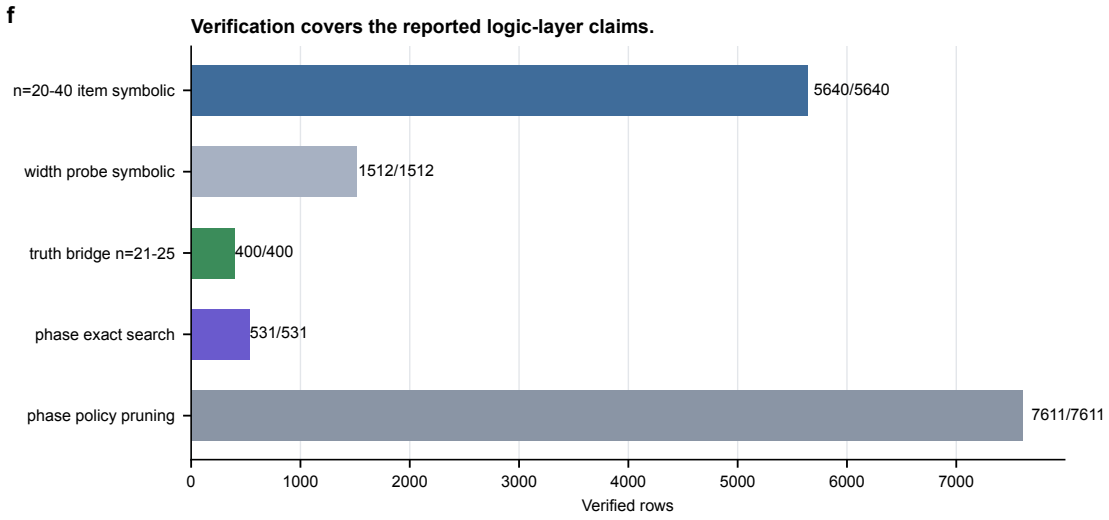

Figure 7: **Verification coverage.** Large term-set experiments are validated symbolically; phase-search and phase-policy rows are checked up to global phase; $n = 21$–25 bridge slices add complete truth table checking on selected generated functions.

Table 39: Scaling and resource audit for high-dimensional experiments. The representative resource mean is method-specific; it is not an average over all baselines in the slice.

Slice	n	Fn./settings	Rows verified	Representative resource mean	Boundary
Large term-set frontier	$24, 28, 32, 40$	96	960/960	score 1090.0; T/CNOT/depth 986/1593/1593; anc. 5.3; time 7.40s	Term-set validation; no full truth table beyond bridge slices.
Stage-gated frontier	$24, 28, 32, 40$	96	96/96	score 1089.7; T/CNOT/depth 985/1593/1593; anc. 5.3; time 10.57s	Controller audit; still term-set symbolic verification.
Action-width probe	$20, 28, 40$	216	1512/1512	score 1130.4; T/CNOT/depth 1023/1648/1648; anc. 5.2; time 1.27s	Sensitivity check; wider root screens are not claimed as better.
Complete truth-table bridge	21–25	40	400/400	score 1126.2; T/CNOT/depth 1019/1647/1647; anc. 5.3; time 4.16s; truth check 0.08s	Complete checking on generated bridge slices only.

Table 40: Verification scope and boundary.

Check	Current coverage	Boundary
$n = 20$–40 item-level symbolic runs	5640/5640	Not full truth-table enumeration.
Width-probe symbolic runs	1512/1512	Supports the default action width, but remains term-set validation.
$n = 21$–25 complete bridge	400/400	Small generated slices; not exhaustive coverage of all high-dimensional functions.
Exact phase-search rows	531/531	Verified up to global phase; no rotation-sequence output.
Phase-policy pruning rows	7611/7611	Learned shortlist rows verified up to global phase; still no rotation-sequence output.

7 Discussion

The evidence supports a specific, bounded claim: Resource-NMCTS is a logical-layer resource-constrained Boolean oracle synthesis method that improves T-count and weighted score over several fixed-representation and external-toolchain baselines. The claim is not that Resource-NMCTS dominates every metric. SSHR remains a strong CNOT-oriented small-function baseline; CirKit AIG/MC remains a strong depth-oriented external probe; and RevKit CLI uses fewer auxiliary lines on most traditional functions.

The ablations also calibrate the role of AI. The neural prior is useful, but the largest gain comes from the searchable algebraic action space and from guarded portfolio selection. This is a better fit for the evidence than claiming that deep reinforcement learning alone drives the result. The Boolean-ring structural audit further separates resource quality from planning cost: at $n = 14$ and $n = 16$ the guard improves score by 0.34–0.82% over the pairwise-wide baseline; at $n = 20$ a deeper screen gives a 7.47% score reduction over the old Resource baseline with an ancilla tradeoff; and the $n = 18$ screen-only row is much faster but worse in score. The sparse depth-frontier audit suggests a simpler planning-cost reduction inside the current frontier: depth 3 was never needed once depth 4 was measured in the audited slices, so a depth-2/4 frontier exactly matched all-depth resources while removing roughly one quarter of frontier evaluation time. This is an empirical controller for the present generated slices, not a proof that depth 3 is universally dominated. The learned sparse depth-4 gate adds a narrower AI-control

result: after the sparse frontier has removed depth 3, the gate skips some depth-4 evaluations without changing score in either the held-out split or the independent multi-seed audit under its conservative threshold. The large, cost-aware, sparse-gated, and staged frontier controllers should be read as operating points on the same search frontier: quality-oriented single-shot selection, faster cost-aware selection, learned sparse-budget control, and a validation-calibrated staged teacher. In the phase/Rz branch, affine and polarity search give verified improvements, and the rank-trained diverse shortlist now tracks the wide-search frontier with 6.25% of the exact evaluations. However, random-repeat margins remain small, so this branch should be read as learned pruning evidence rather than as a solved rotation-synthesis policy. The next target is rotation-spectrum-aware optimization and sequence-level audit.

Several limitations are deliberate. All results are logical-layer estimates; the paper does not model routing, hardware connectivity, native gate sets, noise, or magic-state factories. The ROS-style LUT result is a proxy rather than a full ROS reproduction with SAT garbage management; the line-aware LUT reselection only tests parameter sensitivity within the same proxy. The RevKit CLI baseline is a real exact-oracle reversible-synthesis probe, but CNOT/depth are derived from RevKit's Toffoli-control distribution and the experiment is still not a hardware mapping. High-dimensional symbolic checks validate emitted circuit semantics but do not replace full truth-table enumeration beyond the bridge slices.

8 Conclusion

We presented Resource-NMCTS, a neural Monte Carlo tree search framework for resource-constrained quantum Boolean oracle synthesis over verifiable ANF/FPRM actions. On matched small-function benchmarks, the method substantially lowers T-count and weighted score relative to direct ANF, ESOP, SSHR-H, and ESOP-MILP baselines. On external probes based on ROS-style LUT mapping, mockturtle, CirKit, and legacy RevKit CLI reversible synthesis, the method retains a strong weighted-score advantage while exposing clear depth and ancilla tradeoffs. Large symbolic experiments and $n = 21$–25 complete truth-table bridges support the semantic correctness of the logical-layer circuits. The sparse depth-frontier analysis further reduces high-dimensional planning cost by skipping depth 3 in the audited frontier, and the learned sparse depth-4 gate preserves sparse-frontier score across both held-out and independent-seed audits while removing additional depth-4 evaluations. The staged frontier analysis remains a validation-calibrated controller when depth-4 evaluation itself should be conditional. The next step is not to claim hardware-level optimality, but to strengthen the phase/Rz policy and complete fuller toolchain reproductions while keeping the claim at the logical synthesis layer.

Data and Code Availability

The artifact package used for this manuscript is maintained in the project repository under `resource_nmcts_experiment/`. The directory contains the synthesis code, neural and search-policy scripts, external-toolchain probe drivers, generated raw and summary CSV files, manifest JSON files, LaTeX table fragments, figure source data, and the compiled manuscript PDF. The main reproducibility entry points are the `run_*.py`, `train_*.py`, and `analyze_*.py` scripts listed in the project README; generated measurements are stored in `results/raw_*.csv`, `results/summary_*.csv`, and `results/manifest_*.json`. The paper tables are generated under `paper_latex/tables/`, and the submission figures and source data are stored under `paper_latex/figures/submission_v36/`. The lightweight derived submission package can be rebuilt with `./rebuild_submission_package.sh`; this regenerates paper-facing tables, figures, the submission archive manifest, the uploadable payload archive, audits, and the English PDF from existing experiment artifacts, but does not rerun raw benchmark sweeps, external-toolchain probes, or neural training jobs. The generated payload archive

`submission_package/dist/resource_nmcts_submission_payload.tar.gz` is accompanied by a SHA256 sidecar and a JSON file manifest. The archive manifest records category-level hashes for stable payload groups; terminal submission audits and the compiled PDF are checked separately to avoid self-referential digests. The experiments use the `mcts-qoracle` Conda environment and the direct interpreter `/opt/anaconda3/envs/mcts-qoracle/bin/python`. External-tool rows are reported with manifest-level provenance; hardware mapping, routing, and magic-state-factory artifacts are intentionally absent because they are outside the logical-layer scope of the study.

References

[1] Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[2] David Kremer, Ali Javadi-Abhari, and Priyanka Mukhopadhyay. Optimizing the non-Clifford-count in unitary synthesis using reinforcement learning, 2025. arXiv:2509.21709.

[3] Mulundano Machiya, Matt Menickelly, Paul Hovland, and Ji Liu. MonteQ: A monte carlo tree search based quantum circuit synthesis framework, 2026. arXiv:2604.19029.

[4] Giulia Meuli, Mathias Soeken, Earl Campbell, Martin Roetteler, and Giovanni De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019.

[5] Giulia Meuli, Mathias Soeken, and Giovanni De Micheli. Xor-and-inverter graphs for quantum compilation. *npj Quantum Information*, 8(1):7, 2022.

[6] Giulia Meuli, Mathias Soeken, Martin Roetteler, and Giovanni De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.

[7] Sebastian Rietsch, Abhishek Y. Dubey, Christian Ufrecht, Maniraman Periyasamy, Axel Plinge, Christopher Mutschler, and Daniel D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 824–835. IEEE, 2024.

[8] Jordi Riu, Jan Nogué, Gerard Vilaplana, Artur Garcia-Saez, and Marta P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.

[9] Mathias Soeken and collaborators. CirKit: Logic synthesis and optimization framework. https://github.com/msoeken/cirkit, 2026. CirKit 3 shell and legacy RevKit/CirKit command-line source.

[10] Mathias Soeken and collaborators. mockturtle: Logic network optimization and synthesis. https://mockturtle.readthedocs.io/, 2026. Official-header KLUT-to-XAG probe source.

[11] Mathias Soeken and collaborators. RevKit: Reversible logic and quantum circuit compilation. https://github.com/msoeken/revkit, 2026. Accessed as a local Python API and legacy command-line reversible-synthesis toolchain.

[12] Mathias Soeken, Stefan Frehse, Robert Wille, and Rolf Drechsler. RevKit: An open source toolkit for the design of reversible circuits. In *Reversible Computation*, volume 7165 of *Lecture Notes in Computer Science*, pages 64–76. Springer, 2012.

[13] Mathias Soeken, Heinz Riener, Winston Haaswijk, Eleonora Testa, Bruno Schmitt, Giulia Meuli, Fereshte Mozafari, Siang-Yun Lee, Alessandro Tempia Calvino, Dewmini Sudara Marakkalage, and Giovanni De Micheli. The EPFL logic synthesis libraries, 2018. arXiv:1805.05121.

[14] Dimitrios Tsaras, Antoine Grosnit, Lei Chen, Zhiyao Xie, Haitham Bou-Ammar, and Mingxuan Yuan. ShortCircuit: AlphaZero-driven circuit design, 2024. arXiv:2408.09858.

[15] Robert Wille and Rolf Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of the 46th Annual Design Automation Conference*, DAC '09, pages 270–275. ACM, 2009.

[16] Mingfei Yu, Alessandro Tempia Calvino, Mathias Soeken, and Giovanni De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *2025 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 930–937. ACM, 2025.

[17] Qiang Zheng, Yongzhen Xu, Jiaxi Zhang, Zhaofeng Su, and Shenggen Zheng. CNOT oriented synthesis for small-scale boolean functions using spatial structures of parallelotopes. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2025.

[18] Xiangzhen Zhou, Yuan Feng, and Sanjiang Li. Quantum circuit transformation: A monte carlo tree search framework. *ACM Transactions on Design Automation of Electronic Systems*, 27(6):1–27, 2022.